



Facultad Politécnica, UNA
Departamento de Informática

Algoritmos y Estructura de Datos III

Unidad 1.

Introducción a Java



revisión 2026

OBJETIVOS

- Presentar las construcciones principales de un lenguaje Orientado a Objetos a utilizarse en el curso, Java
- Comparar Java con el Lenguaje C y Python
- Presentar los fundamentos de la Programación Orientada a Objetos
- Servir como material de referencia rápida a usarse en los ejercicios de programación.

Contenido

- [Parte I](#) : Introducción al lenguaje.
- [Parte II](#) : Tipo de datos, operadores, expresiones.
- [Parte III](#) : Entrada/Salida. Estructuras de control.
- [Parte IV](#) : Arreglos y cadenas.
- [Parte V](#) : Introducción a la POO. Clases
- [Parte VI](#) : Herencia
- [Parte VII](#) : Manejo de excepciones
- [Parte VIII](#) : Clases abstractas, interfaces y Generic.
- [Parte IX](#) : Algunas construcciones avanzadas en Java
- [Bibliografía](#)

Parte I

Introducción al lenguaje

- Motivaciones de Java
- Tecnología Java
- Historia
- Ejemplo

Tecnología Java – un vistazo

Java SE 25 (<https://docs.oracle.com/en/java/javase/25/>)

Lenguaje

JCP ([Java Community Process](https://openjdk.org/projects/jcp/))

Es el mecanismo para el desarrollo de las especificaciones para la tecnología Java.

Implementaciones

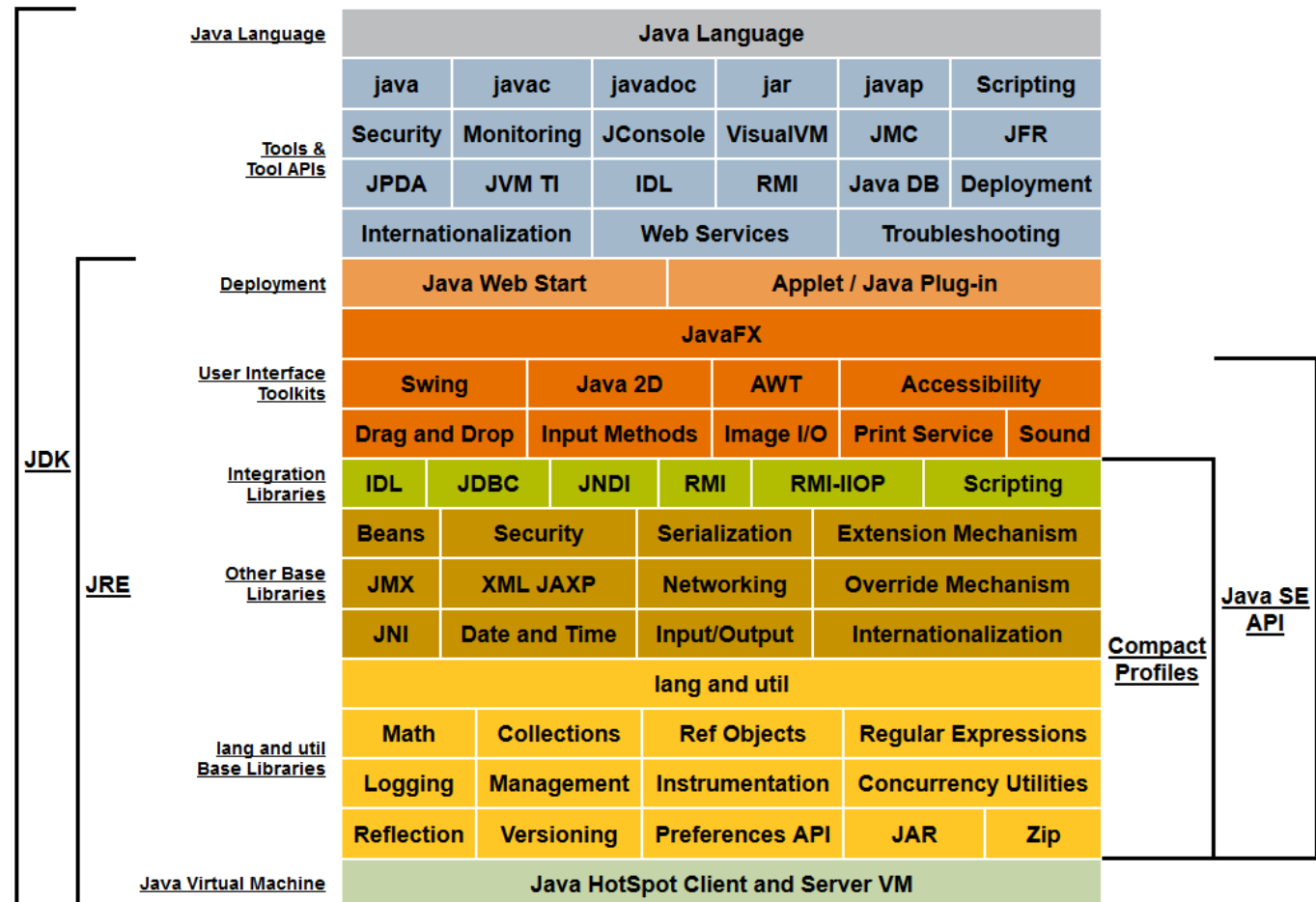
Oracle: JSE JDK 25

<https://docs.oracle.com/en/java/javase/25/>

OpenJDK 25

<https://openjdk.org/projects/jdk/25/>

JDK 7 →



Java es un lenguaje bastante popular

<https://pypl.github.io/PYPL.html>

PYPL PopularitY of Programming Language

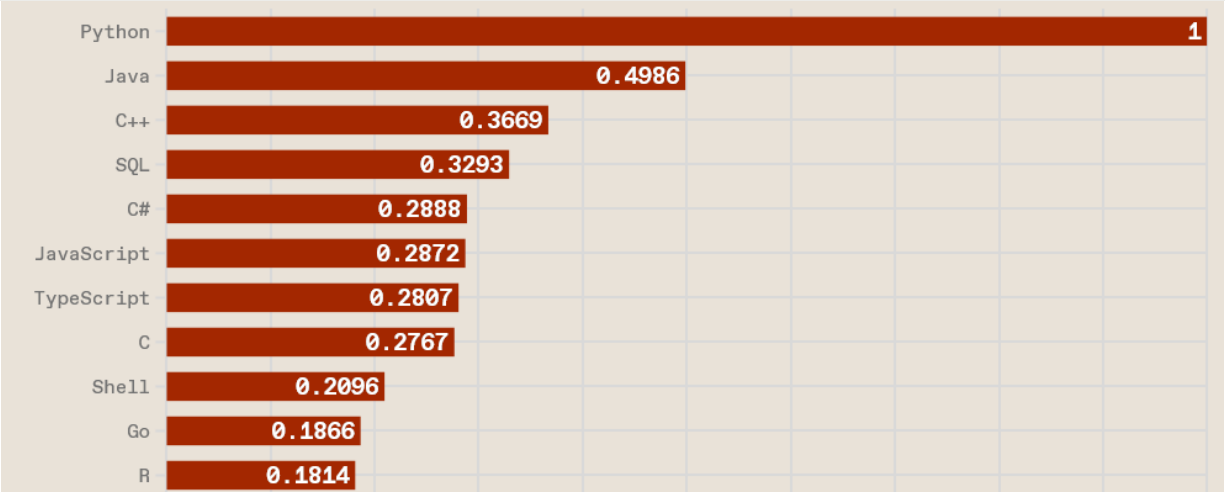
Worldwide, Feb 2026 :

Rank	Change	Language	Share	1-year trend
1		Python	29.97 %	-0.2 %
2	↑↑	C/C++	15.45 %	+8.3 %
3	↓	Java	10.21 %	-4.9 %
4	↑↑	R	6.89 %	+2.4 %
5	↓↓	JavaScript	5.09 %	-2.9 %
6	↑↑↑↑↑	Swift	4.05 %	+1.6 %
7	↑	Rust	3.24 %	+0.1 %
8	↓↓↓	C#	3.19 %	-3.0 %
9	↓↓	PHP	3.19 %	-0.5 %
10	↑↑↑↑↑	Ada	2.93 %	+1.6 %

<https://www.tiobe.com/tiobe-index/>

Feb 2026	Feb 2025	Change	Programming Language	Ratings	Change
1	1		 Python	21.81%	-2.08%
2	4	↑	 C	11.05%	+1.22%
3	2	↓	 C++	8.55%	-2.82%
4	3	↓	 Java	8.12%	-2.54%
5	5		 C#	6.83%	+2.71%
6	6		 JavaScript	2.92%	-0.85%
7	10	↑	 Visual Basic	2.85%	+0.81%
8	15	↑↑	 R	2.19%	+1.14%
9	7	↓	 SQL	1.93%	-0.93%
10	9	↓	 Delphi/Object Pascal	1.88%	-0.29%

IEEE Spectrum / The Top Programming Languages 2025



<https://spectrum.ieee.org/top-programming-languages-2025>

Java : Historia

- Java
 - Ideado en 1991 y oficializado en 1995 por Sun por James Gosling et al. de Sun Microsystems.
(pasó unos cuatro años antes Sun aceptara liderar el proyecto propuesto por un grupo liderado por Gosling)
 - Inicialmente llamado Oak, en honor a un “árbol” que Gosling veía desde de su ventana en Sun, este nombre fue cambiado luego a Java debido a que ya existía un lenguaje con el nombre Oak.
 - En 2006 Java fue liberado bajo GNU GPL
 - En 2009 SUN fue adquirido por Oracle y así también Java.

Java : Historia

- Motivación original para Java
 - La necesidad de un lenguaje de plataforma independiente que debía ser “incrustado” en productos electrónicos de consumo como tostadoras, TVs, etc.
- Primeros pasos
 - El primer proyecto fue un entorno para una especie de PDA denominado Star7
 - En ese momento, WWW e Internet estaban adquiriendo gran popularidad. Se propuso entonces que Java podría ser usado para la programación orientada a Internet.

Lema de los creadores de Java

Write once, run anywhere

Java: motivaciones de su diseño

Extraído del artículo de James Gosling/Henry McGilton
[The Java Language Environment](#) – Mayo 1996

- Simple, orientado a objetos y familiar
- Robusto y seguro
- De arquitectura neutral y portable
- De buen rendimiento
- Interpretado, multihilo y dinámico

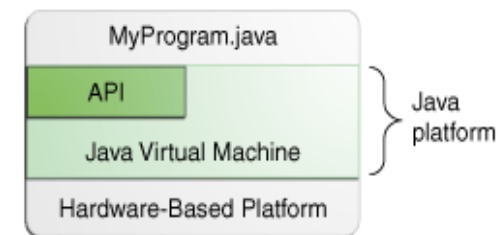
Evolución de las versiones de Java

- Java 1.0 (enero/1996) 8 paquetes, 212 clases
- Java 1.1 (marzo/1997) 23 paquetes, 504 clases
- Java 1.2 (dic/1998) 59 paq., 1520 clases
- Java 1.3 (abr/2000) 77 paq., 1595 clases
- Java 1.4 (2002) 103 paq., 2175 clases
- Java 1.5 (oct/2004) 131 paq., 2656 clases (major PL changes)
- Java 6 (2007) update 45 (Java SE 6)
- Java 7 (2011) update 79(abr/15) (Java SE 7)
- Java 8 (2015) update 401(feb/2024) (Java SE 8) (enhancement in 8: lambda expressions <http://docs.oracle.com/javase/tutorial/java/javaOO/lambdaexpressions.html>)
- Java 9 (2017) actual 9.0.4 (ene/2018) (main enhancement: modules)
- Java 10 (2018) actual 10.0.2 (jul/2018) (main enhancement: inference type)
- Java 11 (2019) actual 11.0.6 (ene/2020) (lambda new features, new HTTP client, WebSocket)
- Java 12 (2019) actual 12.0.2 (jul/2019) (security, soporte idiomas, switch con break implícito, etc)
- Java 13 (2019) actual 13.0.2 (ene/2020) (security, textblocks)
- Java 14 (2020) (registros), Java 15 (2020), Java 16 (2021), Java 17 (2021)
- Java 18 (jul/2022)
- Java 19 (sep/2022)
- Java 20 (Mar/2023) actualmente 20.0.2
- Java 21 (Sep/2023) actualmente 21.0.4 **(LTS)**
- Java 22 (Mar/2024) actualmente 22.0.2
- Java 23 (Sep/2024) actualmente 23.0.2 , Java 24 (Mar/2025) y **Java 25 (Set/25) LTS actualmente 25.0.2**

Details	Status	Documentation	Download	Compare API to
Java 27	DEV	API Lang VM Tools	JDK JRE	26 25 24 23 22 21 20 19 18 17 ...
Java 26	DEV	API Lang VM Tools	JDK JRE	25 24 23 22 21 20 19 18 17 16 ...
Java 25	LTS	API Lang VM Tools	JDK JRE	24 23 22 21 20 19 18 17 16 15 ...
Java 24	EOL	API Lang VM Tools	JDK JRE	23 22 21 20 19 18 17 16 15 14 ...
Java 23	EOL	API Lang VM Tools	JDK JRE	22 21 20 19 18 17 16 15 14 13 ...
Java 22	EOL	API Lang VM Tools	JDK JRE	21 20 19 18 17 16 15 14 13 12 ...
Java 21	LTS	API Lang VM Tools	JDK JRE	20 19 18 17 16 15 14 13 12 11 ...
Java 20	EOL	API Lang VM Tools	JDK JRE	19 18 17 16 15 14 13 12 11 10 ...
Java 19	EOL	API Lang VM Tools	JDK JRE	18 17 16 15 14 13 12 11 10 9 ...
Java 18	EOL	API Lang VM Tools	JDK JRE	17 16 15 14 13 12 11 10 9 8 ...
Java 17	LTS	API Lang VM Tools	JDK JRE	16 15 14 13 12 11 10 9 8 7 ...
Java 16	EOL	API Lang VM Tools	JDK JRE	15 14 13 12 11 10 9 8 7 6 ...
Java 15	EOL	API Lang VM Tools	JDK JRE	14 13 12 11 10 9 8 7 6 5 ...
Java 14	EOL	API Lang VM Tools	JDK JRE	13 12 11 10 9 8 7 6 5 1.4 ...
Java 13	EOL	API Lang VM Tools	JDK JRE	12 11 10 9 8 7 6 5 1.4 1.3 ...
Java 12	EOL	API Lang VM Tools	JDK JRE	11 10 9 8 7 6 5 1.4 1.3 1.2 ...
Java 11	LTS	API Lang VM Tools	JDK JRE	10 9 8 7 6 5 1.4 1.3 1.2 1.1 ...
Java 10	EOL	API Lang VM Tools	JDK JRE	9 8 7 6 5 1.4 1.3 1.2 1.1 1.0
Java 9	EOL	API Lang VM Tools	JDK JRE	8 7 6 5 1.4 1.3 1.2 1.1 1.0
Java 8	LTS	API Lang VM Tools	JDK JRE	7 6 5 1.4 1.3 1.2 1.1 1.0
Java 7	EOL	API Lang VM Tools	JDK JRE	6 5 1.4 1.3 1.2 1.1 1.0
Java 6	EOL	API Lang VM Tools	JDK JRE	5 1.4 1.3 1.2 1.1 1.0
Java 5	EOL	API Lang VM Tools		1.4 1.3 1.2 1.1 1.0
Java 1.4	EOL	API		1.3 1.2 1.1 1.0
Java 1.3	EOL	API		1.2 1.1 1.0
Java 1.2	EOL	API Lang		1.1 1.0
Java 1.1	EOL	API		1.0
Java 1.0	EOL	API Lang VM		
Pre 1.0	EOL			

¿Por qué decimos tecnología JAVA?

- La tecnología JAVA incluye:
 - Un lenguaje de programación
 - Un entorno de desarrollo
 - Un entorno de ejecución
 - Un entorno de distribución
- La plataforma Java incluye:
 - JVM (Java Runtime Environment)
 - Java API (Java Application Programming Interface)



Tecnología Java:

Lenguaje de programación

Java es un lenguaje de programación de propósito general que puede ser usado para crear cualquier tipo de aplicación, como cualquier otro lenguaje de programación convencional. Es concurrente, basado en clases y orientado a objetos. Actualmente con soporte para programación funcional.

Tecnología Java: Entorno de desarrollo

La tecnología Java incluye una serie de herramientas para crear aplicaciones completas:

- un compilador (javac)
- un intérprete (java)
- un generador de documentación (javadoc)
- un conjunto completo de librerías (incluido ED)
- entre otras cosas...

Tecnología Java:

Entorno de ejecución

Las aplicaciones Java son programas que se ejecutan en cualquier máquina donde esté instalado el entorno de ejecución Java (JRE – Java Runtime Environment).

Tecnología Java

Entorno de distribución

- The JRE (*Java Runtime Environment*) que viene con el JDK (Java Development Kit) y contiene un completo conjunto de librerías para aplicaciones básicas, gráficas, web, redes, etc.
- Inicialmente el entorno de distribución era el navegador web. El JVM dejó de usarse en navegadores entre 2015 y 2017 y fue eliminado oficialmente por Oracle en 2018.

Ediciones de Java

- **Java SE** (Standard Edition): aplicaciones cliente desktop.
- **Jakarta EE** (Enterprise Edition): aplicaciones para servidores (ahora gestionado por la [Fundación Eclipse](#))
- **Java ME** (Micro Edition): aplicaciones para dispositivos móviles (casi ya no se usa)
- **Java FX** : desarrollo de aplicaciones gráficas para Internet, para escritorio y para móviles. Actualmente es [OpenJFX](#)

Algunas características de Java:

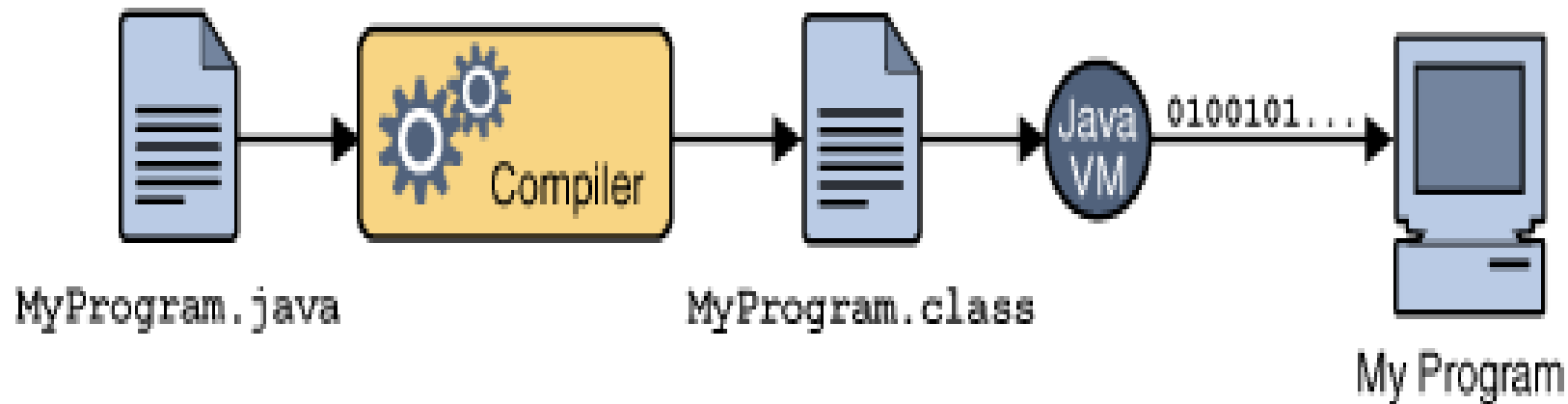
- JVM (Java Virtual Machine): permite portabilidad.
- Colector de basura (Garbage collector)
- 100% Orientado a objetos
- Seguridad en el código ejecutado (verificación del bytecode, control de acceso, sin acceso directo a memoria, sandboxing)
- Manejo de Excepciones
- Concurrencia integrada

COMPARACIÓN ENTRE JAVA, C++ y C

Característica	Java	C++	C
Modelo de ejecución	Bytecode sobre JVM	Código máquina nativo	Código máquina nativo
Portabilidad	Muy alta (JVM)	Media (recompilar)	Media (recompilar)
Gestión de memoria	Automática (GC)	Manual (new/delete) + RAII	Manual (malloc/free)
Riesgo de memory leak	Bajo	Alto si mal gestionado	Alto
Punteros directos	✗ No	✓ Sí	✓ Sí
Aritmética de punteros	✗	✓	✓
Orientación a objetos	Obligatoria (casi todo es clase)	Multiparadigma	No OO
Herencia múltiple	Solo interfaces	✓ Sí	✗
Templates / Generics	Generics (type erasure)	Templates potentes	✗
Concurrencia estándar	API robusta integrada	Desde C++11	No estándar
Manejo de excepciones	Checked y unchecked	No obligatorias	No existen
Seguridad de tipos	Alta	Alta pero flexible	Menor
Acceso a memoria cruda	✗	✓	✓
Control de hardware	Limitado	Alto	Muy alto
Rendimiento máximo teórico	Alto	Muy alto	Muy alto
Curva de aprendizaje inicial	Moderada	Alta	Media
Adecuado para ED inicial	✓ Muy adecuado	✓ Con cuidado	✓ pero complejo

Fases de la creación y ejecución de un Programa Java

- La siguiente figura muestra el proceso de compilar y ejecutar un programa Java



Fases de un Programa Java

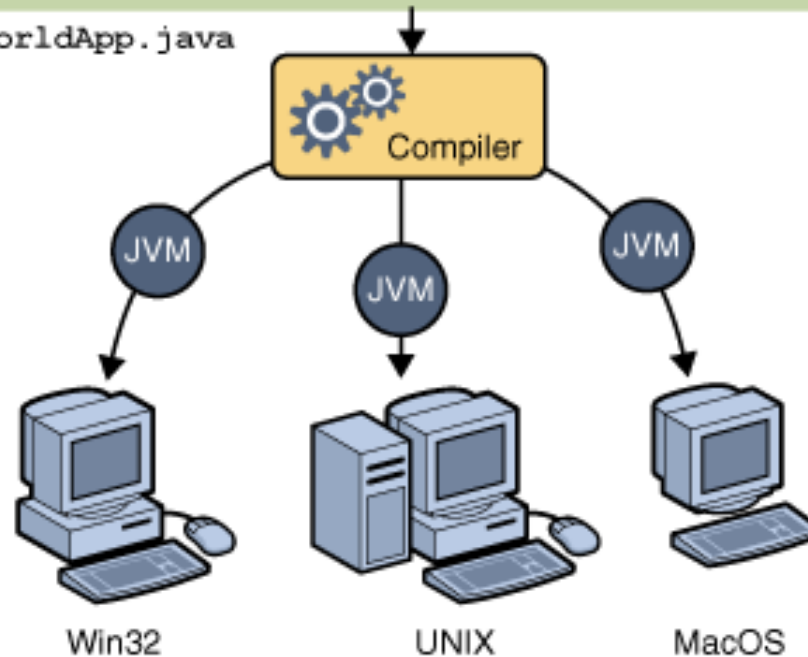
Tarea	Herramienta	Salida
Escribir un programa	Cualquier Editor de Texto	Un archivo con extensión <i>.java</i>
Compilar un programa	Compilador java	Un archivo con extensión <i>.class</i> (java bytecode)
Ejecutar un programa	Intérprete de java	Salida del programa

Java: Un solo código, varias plataformas

Java Program

```
class HelloWorldApp {  
    public static void main(String[] args) {  
        System.out.println("Hello World!");  
    }  
}
```

HelloWorldApp.java



Ejemplo de un código Java

Programa en C que imprime el famoso “*Hola Mundo!!*”

```
#include <stdio.h>

int main {
    printf("Hola Mundo!!\n");
    return 0;
}
```

Compilamos y ejecutamos

```
$ gcc -o test test.c
$ ./test
Hola Mundo!!
$
```

Ahora en Java. El archivo fuente igual que el nombre de la clase: *Test.java*

```
public class Test {

    public static void main ( String [] args ) {
        System.out.println("Hola Mundo!!");
    }
}
```

Compilamos y ejecutamos

```
$ javac Test.java
$ java Test
Hola Mundo!!
$
```


Como obtener el compilador

Oracle:

<https://www.oracle.com/java/technologies/downloads/>

Documentación: <https://docs.oracle.com/en/java/javase/25/>

OpenJDK

<https://jdk.java.net/25/>

Eclipse Temurin JDK <https://adoptium.net/es/temurin/releases>

Amazon Corretto <https://aws.amazon.com/es/corretto/>

Azul Zulu <https://www.azul.com/downloads/?package=jdk#zulu>

REPL (Read-Eval-Print-Loop) en Java

Desde la versión 9 se introdujo una herramienta que permite un uso interactivo del lenguaje, *jshell*

<https://docs.oracle.com/en/java/javase/25/jshell/introduction-jshell.html>

Puede:

- Escribir un programa completo
- Compilar y corregir errores.
- Ejecutar
- Darse cuenta de que está mal
- Editarlo y volver a repetir el proceso
- Tener acceso a la documentación del API



Símbolo del sistema - jshell



```
C:\Users\ccappo>jshell
| Welcome to JShell -- Version 21.0.2
| For an introduction type: /help intro
```

```
jshell> int i = 100
i ==> 100
```

```
jshell> System.out
Signatures:
System.out:java.io.PrintStream
```

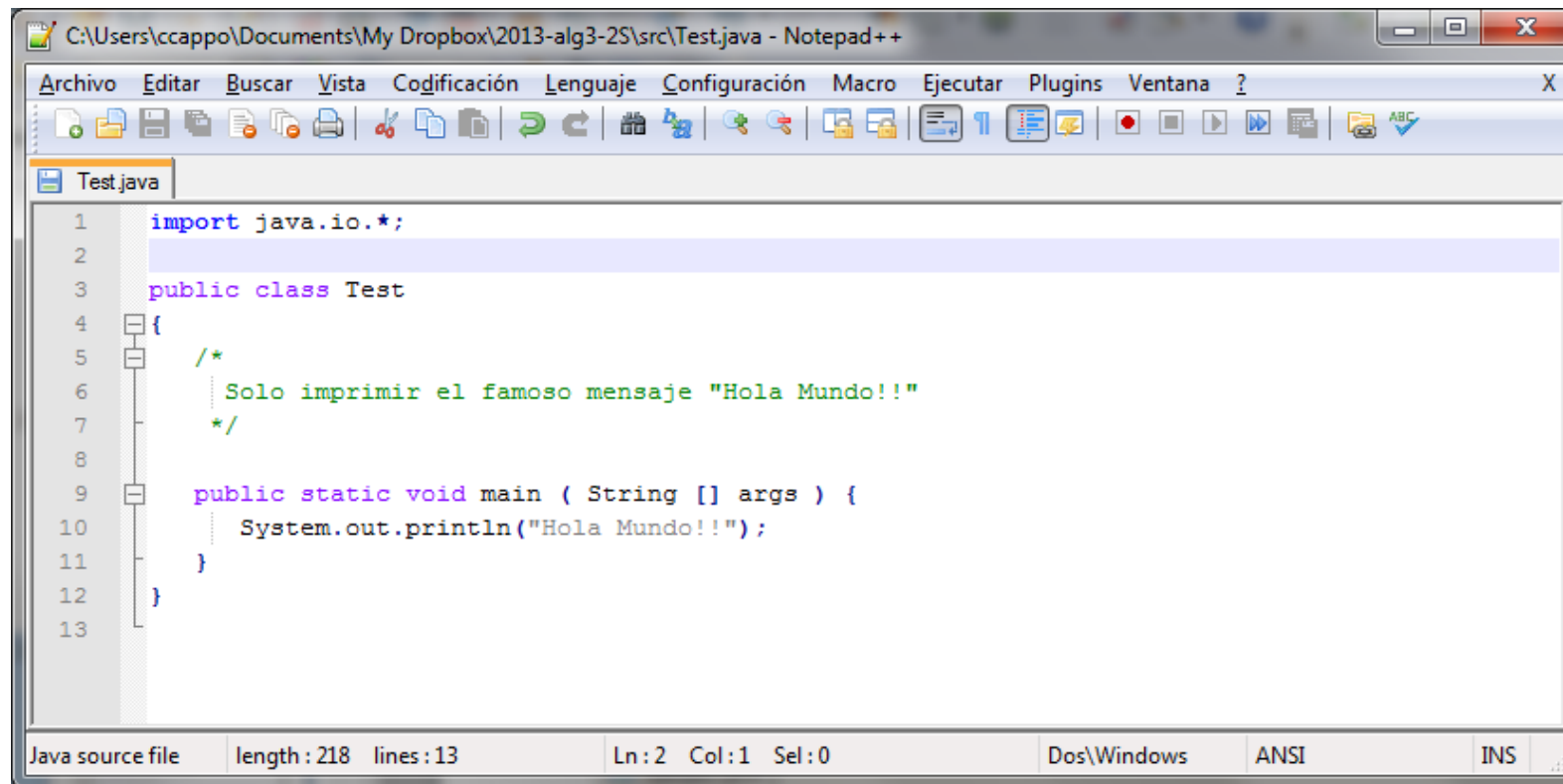
```
<press tab again to see documentation>
```

```
jshell> System.out.println(i)
100
```

```
jshell>
```

Problemas el compilador

Editar el archivo con cualquier editor de texto, usemos un editor denominado Notepad++(<http://notepad-plus-plus.org/>)



```
1  import java.io.*;
2
3  public class Test
4  {
5      /*
6       * Solo imprimir el famoso mensaje "Hola Mundo!!"
7       */
8
9      public static void main ( String [] args ) {
10         System.out.println("Hola Mundo!!");
11     }
12 }
13
```

Java source file length : 218 lines : 13 Ln : 2 Col : 1 Sel : 0 Dos\Windows ANSI INS

Probemos el programa

El compilador es *javac* y el intérprete *java*

```
Símbolo del sistema x + v - □ x
D:\ccappo\Dropbox\2024-alg3-1S\java>javac Test.java
D:\ccappo\Dropbox\2024-alg3-1S\java>java Test
Hola Mundo!!
D:\ccappo\Dropbox\2024-alg3-1S\java>
```

Compilamos (sin errores)

Ejecutamos vía el intérprete

Otros entornos de prueba

jdoodle.com/online-java-compiler-ide/

For simple single file programs and faster execution, use - [Basic Java IDE](#)

MyClass.java

```
1 public class MyClass {
2   public static void main(String args[]) {
3
4     int a = Integer.parseInt(args[0]);
5
6     System.out.println("El valor leído es " + a);
7   }
8 }
9
10
```

Execute Mode, Version, Inputs & Arguments

JDK 17.0.1 Interactive Stdin Inputs

CommandLine Arguments

10

Execute

Result

CPU Time: 0.09 sec(s), Memory: 33592 kilobyte(s) compiled and executed in 1.354 sec(s)

El valor leído es 10

Note: Please check our documentation, or Youtube channel, for more details

Programiz

Online Java Compiler

Interactive Java Course

Main.java

Run

Output

Clear

```
1 // Online Java Compiler
2 // Use this editor to write, compile and run your Java
   code online
3
4 class HelloWorld {
5   public static void main(String[] args) {
6     System.out.println("Hello, World!");
7   }
8 }
```

java -cp /tmp/XeVHruTutZ HelloWorld
Hello, World!

Entorno para aprender

The screenshot shows the CodingBat website for Java practice. The browser address bar displays 'codingbat.com/java'. The page header includes the CodingBat logo, the text 'code practice', and navigation links: 'about', 'help', 'code help+videos', 'done', and 'prefs'. A login section on the right contains input fields for 'id/email' and 'password', a 'log in' button, and links for 'forgot password' and 'create account'. Below the header, there are two tabs: 'Java' (selected) and 'Python'. The main content area displays a grid of eight problem cards, each with a title, a star rating, a description, and a checkmark icon.

Problem	Rating	Description
Warmup-1	★★★★★★★★	Simple warmup problems to get started (solutions available)
Warmup-2	★★★★★★	Medium warmup string/array loops (solutions available)
String-1	★★★★★★★★	Basic string problems -- no loops
Array-1	★★★★★★★★	Basic array problems -- no loops.
Logic-1	★★★★★★★★	Basic boolean logic puzzles -- if else && !
Logic-2	★★★★	Medium boolean logic puzzles -- if else && !
String-2	★★★★★★	Medium String problems -- 1 loop
String-3	★★★★	Harder String problems -- 2 loops

Recomendado para aprender y practicar

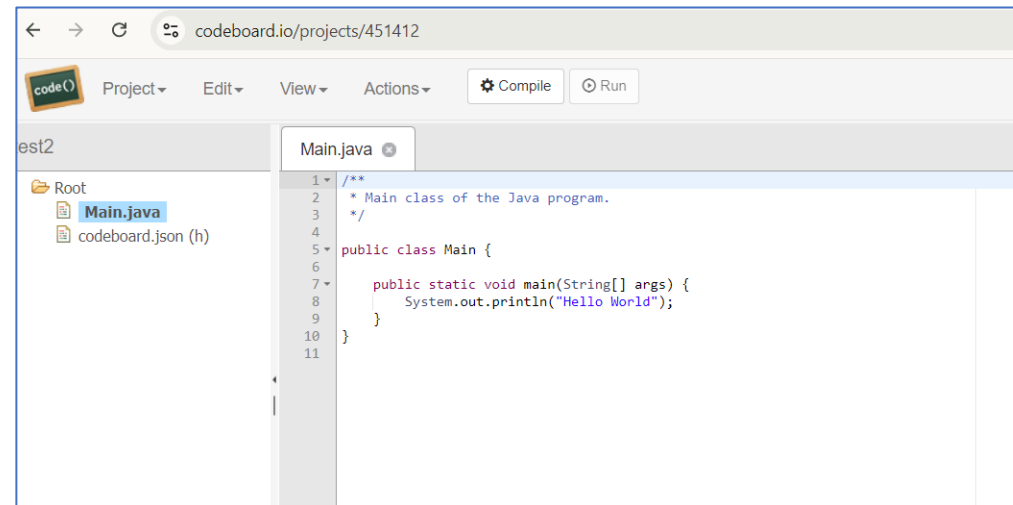
<https://www.geeksforgeeks.org/>

- Entrenamiento con problemas (para diferentes lenguajes, por supuesto para Java).
- Tutoriales interesantes.
- Entrevistas de empleadores de tecnología.
- Java: <https://www.geeksforgeeks.org/java/>

Otros sitios con cursos completos y tutoriales: Udemy, Coursera, JetBrains, Codecademy, edX, javatpoint.com, etc, etc.

Entornos online para Java

- https://www.tutorialspoint.com/online_java_compiler.php (tiene adds) (sharing)
- <https://replit.com> (muy bueno con AI incluido)
- <https://www.programiz.com/java-programming/online-compiler/>
- <https://www.jdoodle.com/online-java-compiler> (tiene AI incluido limitado)
- <https://onecompiler.com/java>
- <https://ide.geeksforgeeks.org/online-java-compiler> (sharing)
- <https://www.codiva.io/> (sharing)
- <https://pythontutor.com/java.html#mode=edit> (muestra objetos en memoria en ejecución)
- <https://codeboard.io/>
- Y muchos más.



Entornos de Desarrollo Integrados (IDE)

- Permiten editar, depurar, ejecutar, autocompletar código, detección de errores temprana, integración con otras herramientas como los gestores de proyectos, control de versiones, entre otros.

- Existen varios para Java

- NetBeans
- Eclipse
- Visual Studio Code
- BlueJ
- IntelliJ IDEA
- JDeveloper
- JCreator



Puede usar cualquier IDE, pero es importante que entienda lo que sucede por debajo, por ello, el curso se enfocará solo en mostrar con un editor y el compilador desde la línea de comandos.

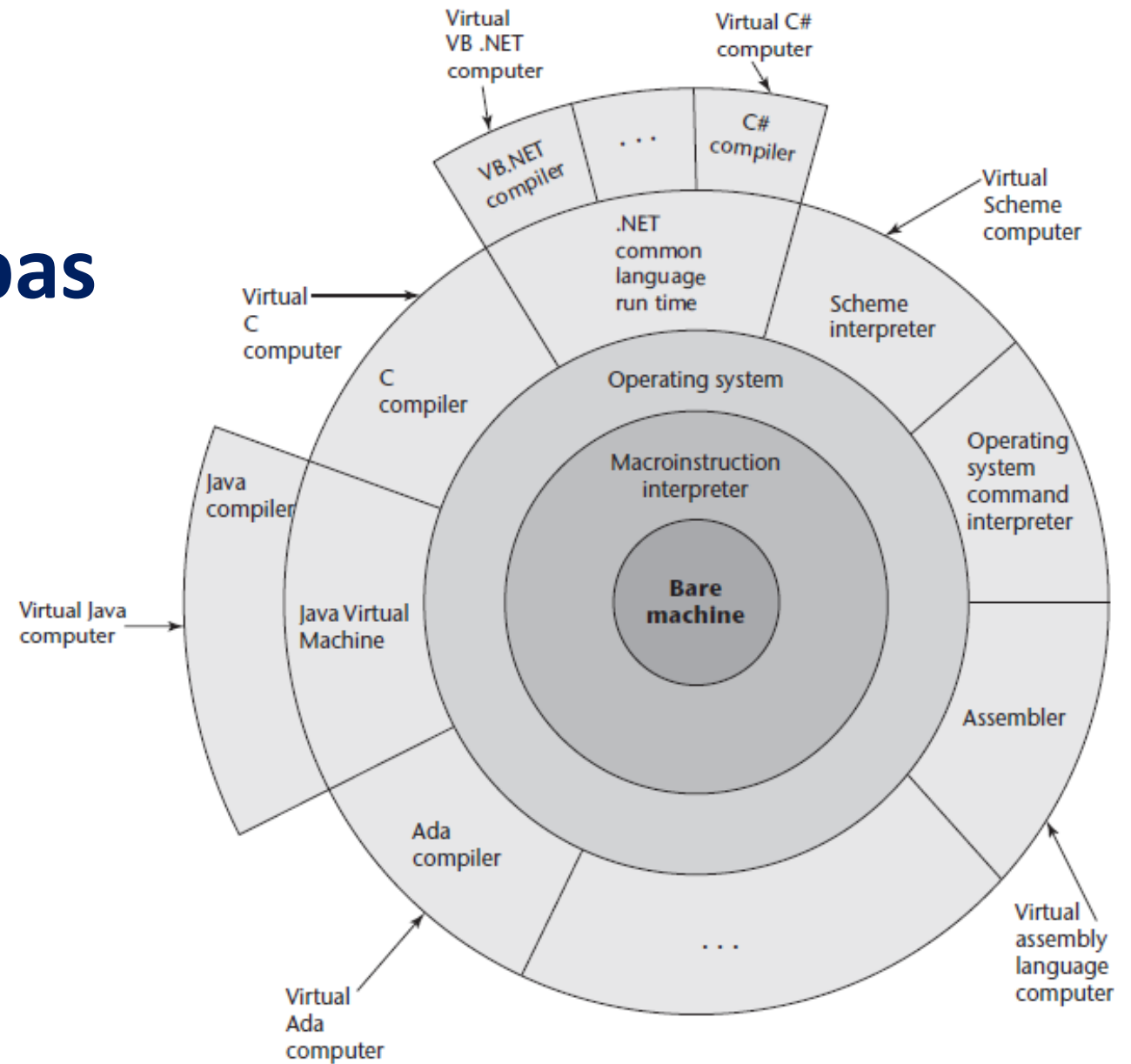
Algunas Diferencias principales con Lenguaje C y Python

Característica	C	Java	Python
Tipo de lenguaje	Orientado a funciones (compilado)	Orientado a Objetos (híbrido)	Orientado a Objetos (interpretado)
Unidad básica de programación	Función, Archivos	Clase = TAD	Script (archivos), Módulos, Clase
Portabilidad del código fuente	Posible con disciplina	Siempre	Siempre
Portabilidad del código compilado	NO, dependiente de la arquitectura	SI (bytecode)	SI (bytecode)
Seguridad	Limitada	Parte del lenguaje	No es parte del lenguaje
Tipo numéricos	Dependiente de la plataforma	Fijos	Fijos siempre como objetos
Tipo booleano	Int y _Bool	boolean	Bool
Tipo caracter	ASCII	Unicode	Unicode
Dirección de memoria	Puntero	Referencia	Referencia
Arreglos	Estáticos (estáticos de stack)	Dinámicos de Heap con tamaño estático	Listas dinámicas de elementos no homogéneos
Métodos de obtener y liberar memoria	<i>malloc()</i> y <i>free()</i>	<i>new</i> y liberación automática (garbage collector)	Automático con la asignación y liberación automática (GC)
Conversión explícita	Siempre es posible	Se controla	Es posible
Conversión implícita	Siempre	Debe ser explícita si es con pérdida de precisión	Es automática siempre
Cadenas	Por convención	Incluido como tipo inmutable	Incluido como inmutable
Asignación de memoria para estructura de datos y arreglos	Heap, Stack, Data	Heap	Heap

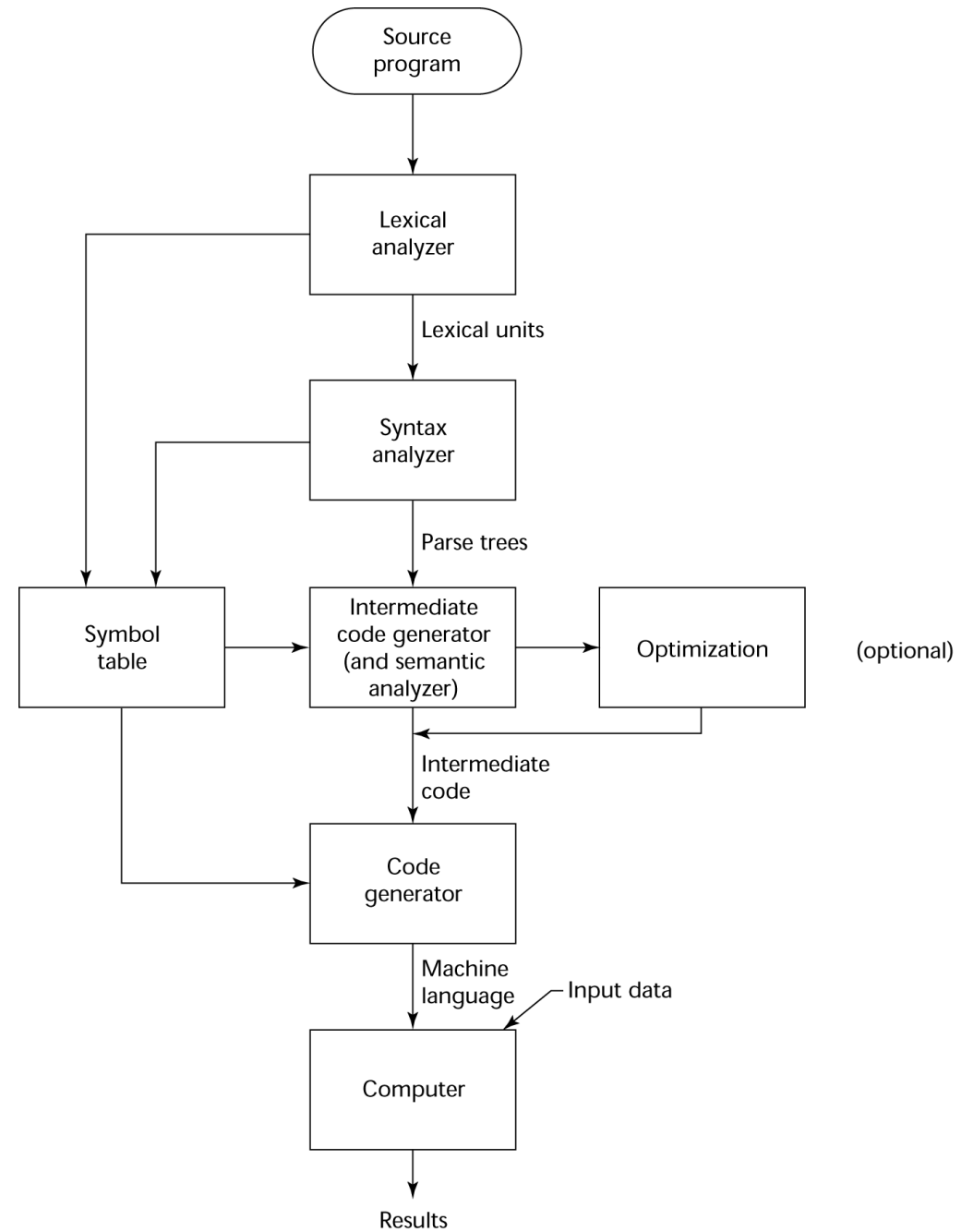
Lenguajes de Programación

- **Sintaxis:** la forma o estructura de las expresiones, sentencias y unidades de programas.
- **Semántica:** el significado de las expresiones, sentencias y unidades del programa.
- La sintaxis y la semántica proveen la **definición del lenguaje**.
- **Usuarios:** diseñadores de lenguajes, implementadores del lenguaje, programadores que usan el lenguaje.
- **Implementaciones:** compiladores, intérpretes e híbridos.
- **Algunas clasificaciones:**
 - **Modelo computacional:** imperativos, funcionales (List, Haskell) y lógicos (Prolog).
 - **Dominios:** científicos (Fortran, R, Matlab), negocios (Cobol, Java, etc), IA (Python, R, Matlab, etc), sistemas (C, C++, Rust), web (PHP, Java, JS), generales (Java, Python, C++), etc.
 - **Metodología:** estructurados/procedurales, orientados a objeto (datos), concurrentes/paralelos.

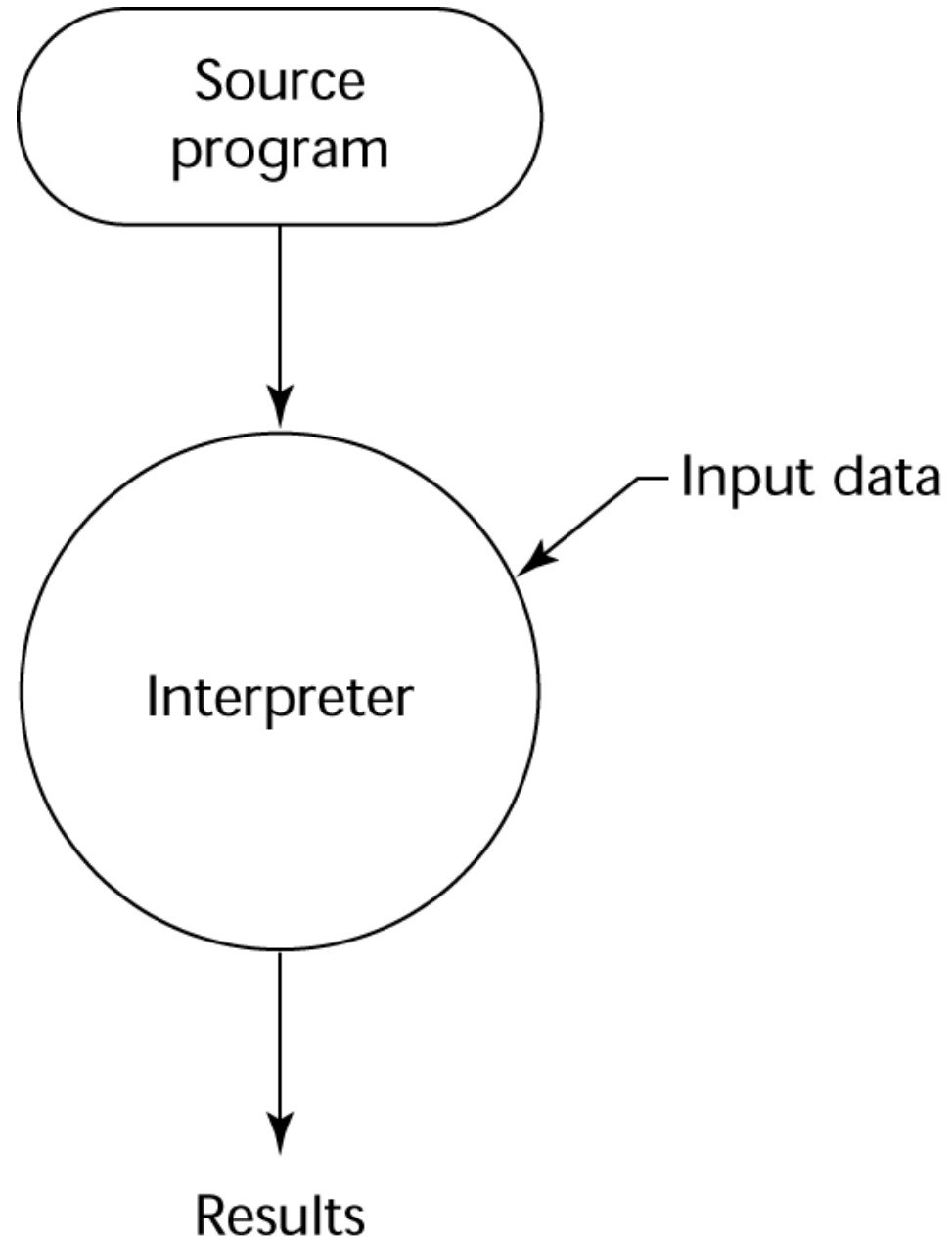
Modelo típico en capas de un sistema computacional



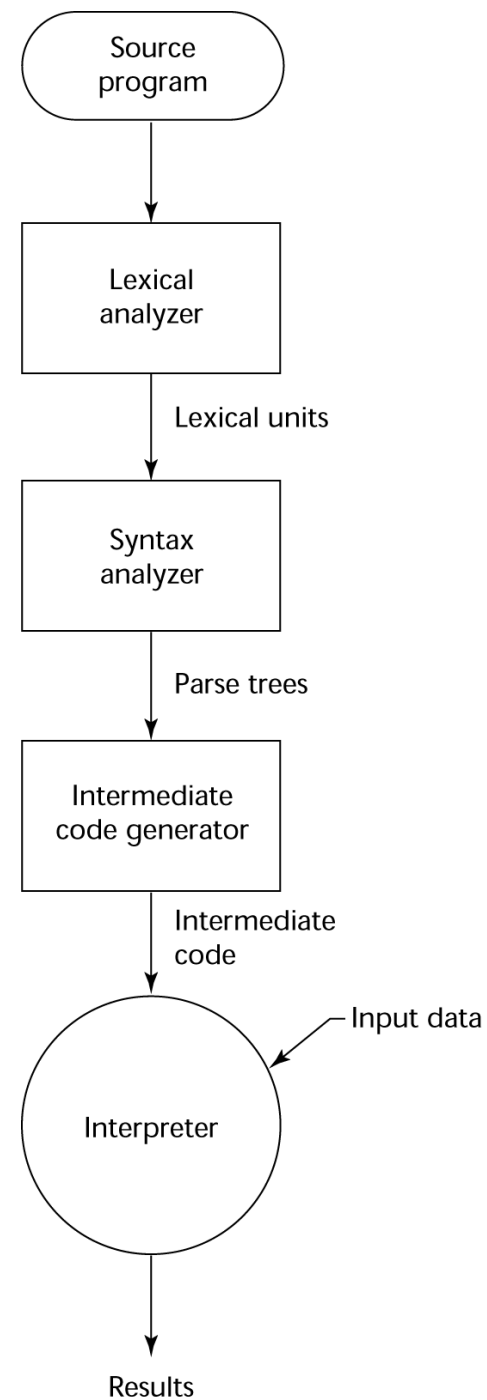
Compiladores



Intérpretes puros



Híbridos (compilado e interpretado)



Algunas características semánticas

- Sintaxis es muy similar a C y C++, de hecho, casi iguales.
- Tipo de datos primitivos se mantienen entre plataforma y plataforma.
- Comentarios al estilo C y C++:
 - `//` unilínea
 - `/*` multilínea `*/`

Parte II

Tipo de datos, operadores, expresiones

- Programa ejemplo
- Literales, identificadores, palabras reservadas
- Comentarios en Java
- Tipo de datos
- Variables: declaración e inicialización
- Sentencias y bloques de sentencias
- Salida de datos
- Operadores: aritméticos, relacionales, lógicos.

Ejemplo, un repaso

```
1 import java.io.*;

2 public class Test {
3     /**
4      * Mi Primer programa Java
5      */
6     public static void main( String[] args ){
7         // Imprimo el mensaje "Hola Mundo!!"
8         System.out.println("Hola Mundo!!");
9         // Puedo hacerlo tambien como en C ;)
10        System.out.printf("%s", "Hola Mundo!!");
11    }
12}
```

Ejemplo...

```
1  import java.io.*;
```

```
2  public class Test{
```

- Una declaración ***import*** permite que un nombre de tipo pueda ser referido por un nombre simple (sin la especificación completa).
- En este ejemplo no es estrictamente necesario el uso de import ya que no tenemos ningún tipo que comience con *java.io.** (aquí mostramos solo para explicar el sentido de ***import***)

Ejemplo...

```
1 import java.io.*;
```

```
2 public class Test{
```

- Indica que el nombre de la clase será **Test**
- En Java, todo el código debe ser puesto dentro de la declaración de una clase.
- La clase utiliza un modificador de acceso “**public**”, que indica que la clase es accesible desde otras clases del mismo paquete (*paquete=es una colección de clases agrupadas en un directorio*).

Ejemplo..

```
1  import java.io.*;

2  public class Test {
3      /**
4       * Mi Primer programa Java
5       */
6      public static void main( String[] args ){
```

- Indica que el nombre de un método en **Test** que se llama **main**.
- El método **main** es el punto de entrada de un programa Java.
- Asegurarse de que el “*signature*” o “*firma*” del método tenga esta forma.

Ejemplo..

```
1 import java.io.*;

2 public class Test {
3     /**
4      * Mi Primer programa Java
5      */
6     public static void main( String[] args ){

7         // Imprimo el mensaje "Hola Mundo!!"
8         System.out.println("Hola Mundo!!");
```

`System.out.println()`, imprime el texto encerrado entre comillas dobles en la salida estándar con un enter al final

Ejemplo..

```
1  import java.io.*;

2  public class Test {
3      /**
4       * Mi Primer programa Java
5       */
6      public static void main( String[] args ){

7          // Imprimo el mensaje "Hola Mundo!!"
8          System.out.println("Hola Mundo!!");
9          // Puedo hacerlo tambien como en C ;)
10         System.out.printf("%s", "Hola Mundo!!");
11     }
12 }
```

`System.out.printf()`, igual que *printf* de C

Repasando..

- Recordar que:
 - La extensión de los archivos fuentes es **.java**
 - El nombre del archivo siempre debe coincidir con el nombre de la clase.
 - Conviene siempre que nuestro código tenga comentarios útiles

Java: *sintaxis y otras cosas*

- Recordar que mayúsculas y minúsculas son diferentes, así *HoLa* es diferente a *HOLA*, *hola*, etc.
- Las sentencias siempre terminan en ; (punto y coma) igual que C y C++.
- Un bloque es un conjunto de sentencias que se inicia con un { y termina con un }.
- Java es de formato libre, es decir se puede “jugar” con los espacios, aunque es recomendable mantener siempre un estilo de programación

Java: *identificadores*

- Se utiliza para nombrar variables, métodos, nombre de clases, etc.
- Deben empezar siempre con una letra, el signo _ o el signo \$.
- No pueden utilizarse palabras reservadas (keywords)
- Se recomienda:
 - Que los nombres de *clase* empiecen con Mayúscula.
 - Que los nombres de *métodos* y *variables* empiecen con minúsculas. Si es un nombre de método tiene dos palabras, el inicio de la siguiente palabra es en mayúscula. Por ejemplo: *esPrimo*, *agregarArista*, *etc.*

Java: *palabras reservadas*

Palabras clave en Java

abstract	assert	boolean	break	byte
case	catch	char	class	continue
default	do	double	else	enum
extends	final	finally	float	for
if	implements	import	instanceof	int
interface	long	native	new	package
private	protected	public	return	short
static	strictfp	super	switch	synchronized
this	throw	throws	transient	try
void	volatile	while		

Palabras clave que no se utilizan actualmente

const	goto
-------	------

Java : *literales*

- Del tipo enteros, pueden ser en diferentes “bases”.
 - Decimal : normal: **12121**, **12812**. Es del tipo *long* con el sufijo L, así **10L** es long no int.
 - Hexadecimal : precedido por 0x ó 0X . **0xAB**
 - Octal, precedido por 0 (cero): **014**
- Del tipo punto flotante
 - **583.45** (estandar) ó **5.8345e2** (científico)
 - **0.0d** (double) ó **0.0f** (float)
- Del tipo lógico
 - **true** o **false** (en minúsculas)

Java: literales (cont.)

- Del tipo caracter
 - Encerrados entre comillas simples 'a', 'b' '\n', '\b' (con secuencia de escape como C y C++)
 - *Obs: son del tipo UNICODE (16 bits) no ASCII*
- Cadenas (En Java no es primitivo pero tiene un tratamiento especial, el tipo de dato es *String*)
 - Secuencia de caracteres encerrados entre comillas dobles. Por ejemplo “Algoritmos y ED III”.

Java : tipo de datos primitivos – dominio

Tipo	Tamaño en bits	Valores	Estándar
boolean		true o false	
[Nota: una representación boolean es específica para la Máquina virtual de Java en cada plataforma].			
char	16	'\u0000 ' a '\uFFFF ' (0 a 65535)	(ISO, conjunto de caracteres Unicode)
byte	8	-128 a +127 (-2^7 a $2^7 - 1$)	
short	16	-32,768 a +32,767 (-2^{15} a $2^{15} - 1$)	
int	32	-2,147,483,648 a +2,147,483,647 (-2^{31} a $2^{31} - 1$)	
long	64	-9,223,372,036,854,775,808 a +9,223,372,036,854,775,807 (-2^{63} a $2^{63} - 1$)	
float	32	<i>Rango negativo:</i> -3.4028234663852886E+38 a -1.40129846432481707e-45 <i>Rango positivo:</i> 1.40129846432481707e-45 a 3.4028234663852886E+38	(IEEE 754, punto flotante)
double	64	<i>Rango negativo:</i> -1.7976931348623157E+308 a -4.94065645841246544e-324 <i>Rango positivo:</i> 4.94065645841246544e-324 a 1.7976931348623157E+308	(IEEE 754, punto flotante)

Java: *variables*

- Una variable es un ítem de dato asociado a un objeto en memoria.
- Tiene siempre un tipo de dato y un identificador o nombre.
- Declarar una variable

`<tipo_dato> <identificador> [ = <valor_inicial> ]`

Java: *tipos de variables*

- En Java existen dos tipos de variables
 - Primitivas
 - Referencias
- **Primitivas:** asociadas a tipos primitivos
 - Ejemplo: `int a`, `float b`, `boolean esPar`;
- **Referencia:** es un puntero (fijo) que se asocia a un dato en el HEAP (área de datos de la memoria). Asociado a objetos (de clases) y arreglos.

```
String cad = "Hola"    //cad es del tipo referencia.
```

Java: *algunos operadores*

- Tipos de operadores disponibles:
 - Aritméticos:
 - Binarios: *, /, +, -, %
 - Unarios: ++, --, -
 - Relacionales: >, <, <=, >=, ==, !=, instanceof
 - Bits: <<, >>, >>>, &, |, ^, !
 - Lógicos: && (and en sc), & (and), || (or en sc), |, ! (not)
 - Condicionales: ?:
 - Asignación: =, +=, -=, *=, /=, %=, ..

Operadores ordenados por precedencia

Operador	Descripción	Asociatividad
++ --	unario de postincremento unario de postdecremento	de derecha a izquierda
++ -- + - ! ~ (tipo)	unario de preincremento unario de predecremento unario de suma unario de resta unario de negación lógica unario de complemento a nivel de bits unario de conversión	de derecha a izquierda
* / %	multiplicación división residuo	de izquierda a derecha
+ -	suma o concatenación de cadenas resta	de izquierda a derecha
<< >> >>> < <= > >= instanceof	desplazamiento a la izquierda desplazamiento a la derecha con signo desplazamiento a la derecha sin signo menor que menor o igual que mayor que mayor o igual que comparación de tipos	de izquierda a derecha

Operador	Descripción	Asociatividad
==	es igual a	de izquierda a derecha
!=	no es igual a	
&	AND a nivel de bits AND lógico booleano	de izquierda a derecha
^	OR excluyente a nivel de bits OR excluyente lógico booleano	de izquierda a derecha
	OR incluyente a nivel de bits OR incluyente lógico booleano	de izquierda a derecha
&&	AND condicional	de izquierda a derecha
	OR condicional	de izquierda a derecha
?:	condicional	de derecha a izquierda
=	asignación	de derecha a izquierda
+=	asignación, suma	
-=	asignación, resta	
*=	asignación, multiplicación	
/=	asignación, división	
%=	asignación, residuo	
&=	asignación, AND a nivel de bits	
^=	asignación, OR excluyente a nivel de bits	
=	asignación, OR incluyente a nivel de bits	
<<=	asignación, desplazamiento a la izquierda a nivel de bits	
>>=	asignación, desplazamiento a la derecha a nivel de bits con signo	
>>>=	asignación, desplazamiento a la derecha a nivel de bits sin signo	

Repasando

Librerías/Utilidades/Funciones incorporadas ya vistas

- Entrada/Salida
 - Salida: **System.out.println**, **System.out.print**, **System.out.printf**
 - Entrada: Clase **Scanner**
- Matemática (clase **Math**) (fácil de verificar vía *JShell*)
 - **Math.sqrt**, **Math.log**, **Math.pow**, **Math.exp**, etc.

```
public class Math {  
    ...  
    double sqrt ( double a )  
    ...  
}
```

Diagram illustrating the components of the **Math** class and its **sqrt** method signature:

- Math**: Nombre de la Librería (como clase)
- double**: Tipo de retorno
- sqrt**: Nombre del método o función
- (double a)**: Firma de la función (como miembro de clase) / Lista de parámetros

Parte III

Entrada/Salida y Estructuras de Control

- Argumentos de la línea de comandos
- Lectura de datos desde la consola
- Estructuras de control

Java: *conversiones*

- **Conversión explícita:** a través de métodos/funciones específicas (`Math.round: double->int`, `Integer.parseInt: String->int`, `Double.parseDouble: String->double`, etc)
 - Cada tipo primitivo tiene su Clase equivalente (*wrapper class*)
- **Conversión automática:** para tipos primitivos y solo cuando es “legal” (de un tipo de rango menor a otro de rango mayor)
- ***Casting*** (conversión explícita): requerida por el programador y que implica pérdida de precisión.
- Conversión automática para String: un tipo puede ser convertido a String automáticamente vía +

```
System.out.println("Hola" + 10.54f + !true);
```

Clases *wrapper*(envoltorios)

Tipo Primitivo	Clase wrapper
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
boolean	Boolean
char	Character

Argumentos de la línea de comandos

```
/**
 * Leer un parámetro entero desde la línea de comandos e imprimir
 * su cuadrado.
 */

public class Test{
    public static void main( String[] args ){

        int n = Integer.parseInt(args[0]);

        System.out.printf("El cuadrado de %d es %.0f", n, Math.pow(n,2));

    }
}
```

Uso de parámetros de línea de comandos

\$java Test **10**

args = { "10" }

Lista de parámetros como un vector de cadenas

int **n** = Integer.parseInt(args[0]);

El cuadrado de 10 es 100

Argumentos de la línea de comandos

Ejemplo 2:

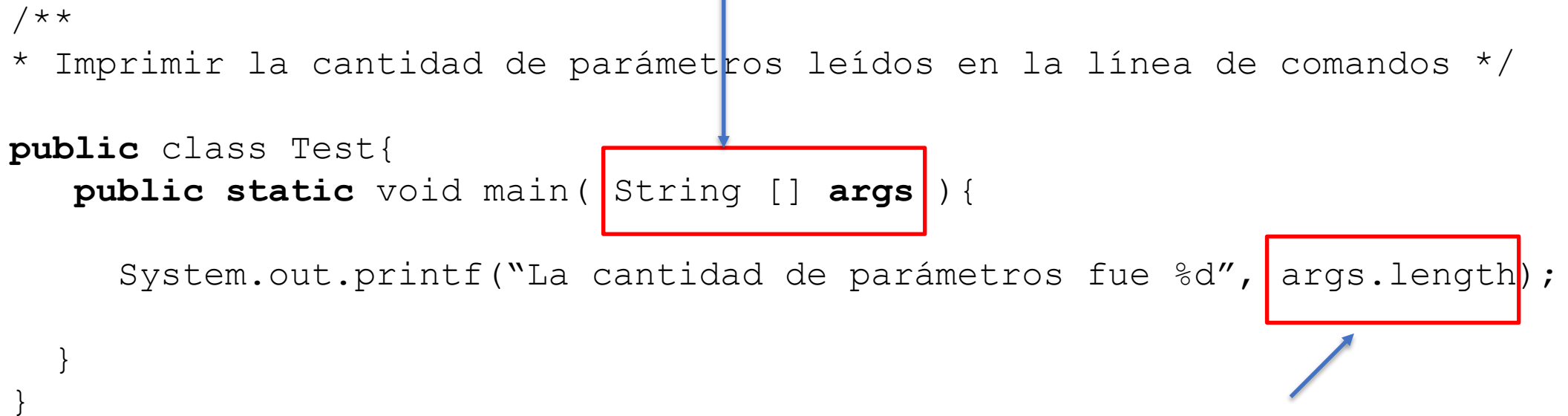
[] indica que es un vector/arreglo (un tipo de dato estructurado)

```
/**
 * Imprimir la cantidad de parámetros leídos en la línea de comandos */

public class Test{
    public static void main( String [] args ) {

        System.out.printf("La cantidad de parámetros fue %d", args.length);

    }
}
```



Un vector tiene como *propiedad* la cantidad de elementos que tiene denominado **length**

Uso de parámetros de línea de comandos

Lista de parámetros como un vector de cadenas

`args = { "10", "20", "30", "40", "50" }`

`$java Test 10 20 30 40 50`

La cantidad de parámetros fue 5

Lectura desde la consola

```
import java.util.*;
```

```
...
```

```
.. class ..{
```

```
    private static int leer_int() {  
        int n;  
        Scanner s = new Scanner(System.in);  
        n = s.nextInt();  
        return n;  
    }
```

```
}
```

Obtener siguiente entero.

Otros: nextLong, nextFloat, nextLine(),
nextDouble, nextBoolean, etc

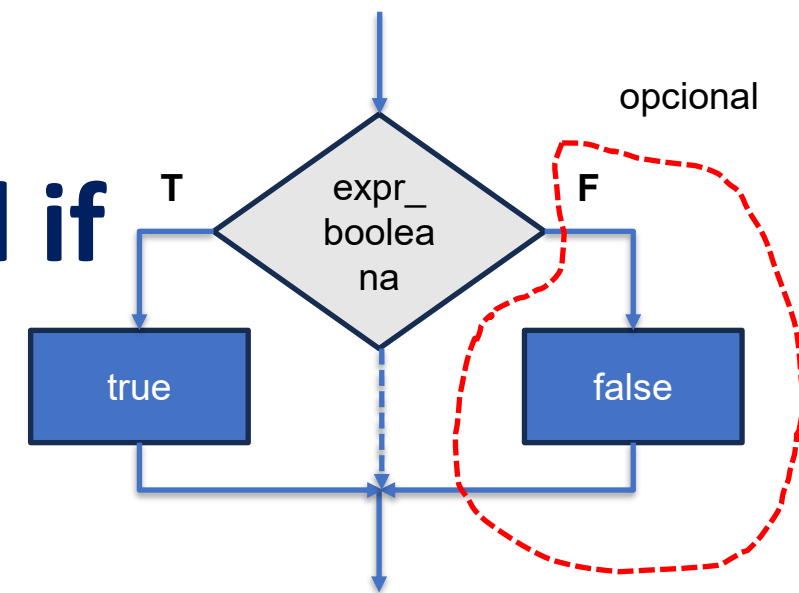
Estructuras de control

- Una **estructura de control** en una especificación sintáctica que consta de un mecanismo de control y las sentencias asociadas a la estructura. Se utiliza cuando el flujo de ejecución del programa no es lineal y se basa en condiciones del estado de las variables.
- Dos posibles tipos:
 - **Condicionales o de decisión** :
Permiten *seleccionar* la ejecución de código
(if, if-else, switch)
 - **Iteración**:
Permiten ejecutar *varias veces* porciones de código
(while, for, do-while).

Estructura de control condicional if

```
//Se ejecuta <sentencia_T> o <bloque_sentencias_T>  
//cuando <expr_booleana> es true  
if ( <expr_booleana> )  
    <sentencia_T> | <bloque_sentencias_T>;
```

```
//Se ejecuta <sentencia_T> o <bloque_sentencias_T>  
//cuando <expresion_booleana> es true  
//Se ejecuta <sentencia_F> o <bloque_sentencias_F>  
//cuando <expresion_booleana> es false  
if ( <expr_booleana> )  
    <sentencia_T> | <bloque_sentencias_T>  
else  
    <sentencia_F> | <bloque_sentencias_F>
```



Una <expr_booleana> es una expresión que retorna **true** o **false**.

Puede ser:

- Una expresión relacional (>, ==, !=, >=, <=, <)
- Una expresión lógica (||, &&, !)
- Una variable booleana o constante booleana.
- Una **combinación** de las anteriores

Un **<bloque_sentencias>** es una secuencia de sentencias encerradas entre llaves { } 71

Estructuras de control condicional, ejemplo

Imprimir un mensaje si una persona es adulta de acuerdo a su edad

```
if ( edad >= 18 )  
    System.out.println("Es una persona adulta")
```

Imprimir un mensaje si una persona es adulta de acuerdo a su edad. Si no lo es imprimir que es menor de edad.

```
if ( edad >= 18 )  
    System.out.println("Es una persona adulta")  
else  
    System.out.println("Es una persona menor de edad")
```

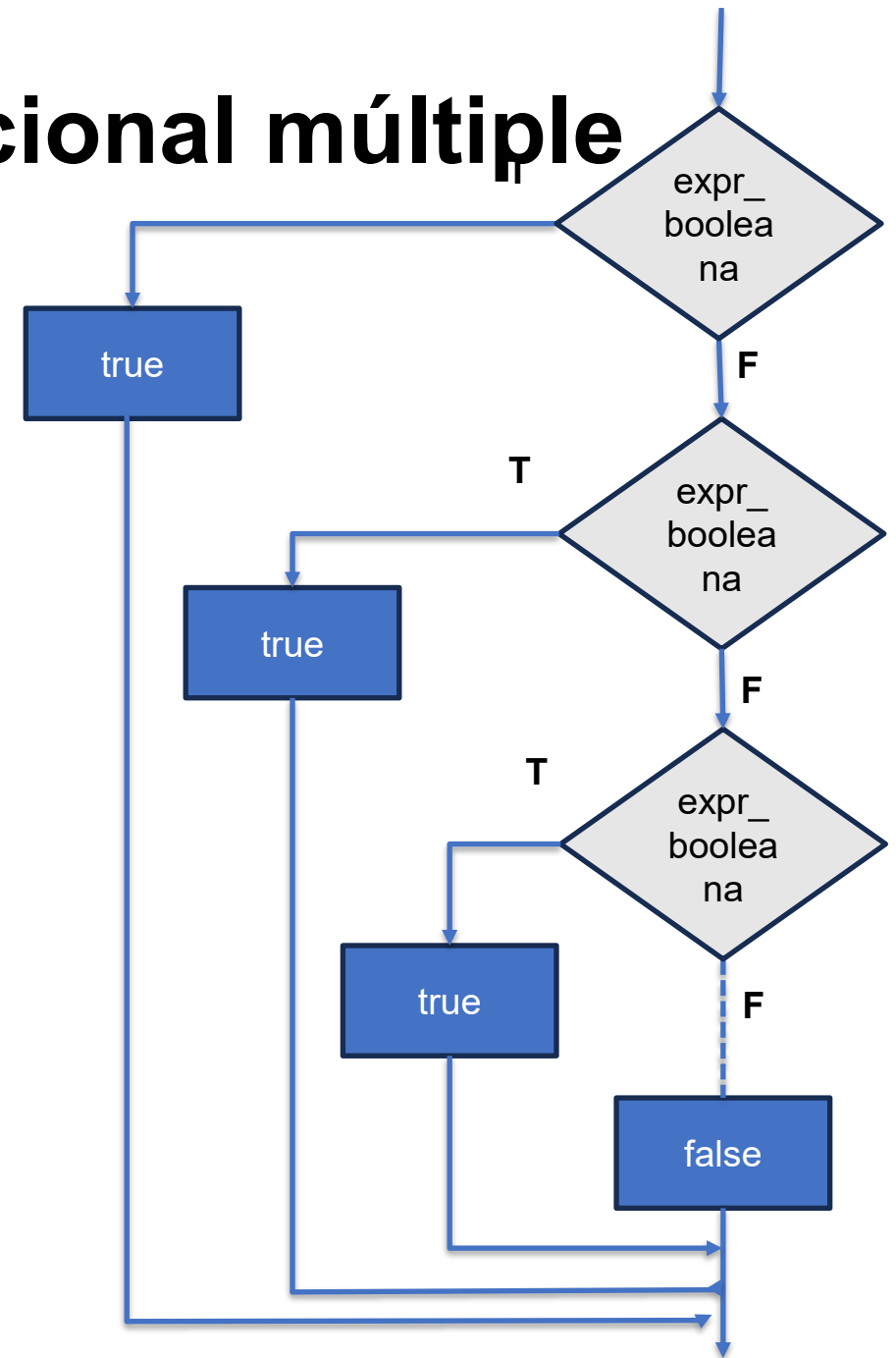

Estructuras de control condicional, ejemplos

Problema	Código
Valor absoluto	<pre>if (n < 0) n = -n;</pre>
Colocar x e y en orden	<pre>if (x > y) { int tmp = x; x = y; y = tmp; }</pre>
Máximo entre x e y	<pre>if (x > y) max = x; else max = y;</pre>
Chequeo en división	<pre>if (den == 0) System.out.print("División por cero") else System.out.print("El cociente es %f", num/den);</pre>
Chequeo en el cálculo de una ecuación cuadrática	<pre>double discriminante = b*b - 4*a*c; if (discriminante < 0) System.out.println("No existen raíces reales"); else { System.out.printf("x1=%f\n", (-b + Math.sqrt(discriminante))/(2*a)); System.out.printf("x2=%f\n", (-b - Math.sqrt(discriminante))/(2*a)); }</pre>

Pruebe en **JShell** el funcionamiento

Estructuras de control condicional múltiple

```
if ( <expr_booleana> )  
    <sentencia> | <bloque_sentencias>  
  
else if ( <expr_booleana> )  
    <sentencia> | <bloque_sentencias>  
  
else if ( <expr_booleana> )  
    <sentencia> | <bloque_sentencias>  
..  
  
else  
    <sentencia> | <bloque_sentencias>
```



Estructuras de control condicional

Problema	Código
Imprimir si la variable leída es positiva, negativa o cero.	<pre>if (n > 0) System.out.println("Es positivo"); else if (n < 0) { System.out.printf("Es negativo"); else System.out.printf("Es cero");</pre>
Determinar la nota de un alumno de acuerdo a su puntaje: De 0 a 59 es 1 De 60 a 69 es 2 De 70 a 79 es 3 De 80 a 90 es 4 De 81 a 100 es 5 Asumir que puntaje es correcto (esta entre 0-100)	<pre>if (puntaje <= 59) nota = 1; else if (puntaje <= 69) nota = 2; else if (puntaje <= 79) nota = 3; else if (puntaje <= 90) nota = 4; else nota = 5;</pre>

Pruebe en **JShell** el funcionamiento

Estructura condicional múltiple switch

Permite la selección de varios bloques según el contenido de una expresión “entera” o una cadena (desde la versión 7). Es parecido al del lenguaje C. Es posible siempre utilizar los **if** múltiple en vez de esta estructura.

```
switch ( <expresion_entera_cadena_enum> ) {  
    case <constante_entera_cadena_1_enum_1>:  
        <sentencia_1>; //  
        <sentencia_n>; //bloque 1  
        break;  
    case <constante_entera_cadena_2_enum_2>:  
        <sentencia_1>; //  
        <sentencia_n>; //bloque 2  
        break;  
    default:  
        <sentencia_1>; //  
        <sentencia_n>; //bloque n  
}
```

Estructura de control condicional switch

- Java evalúa primero la expresión del switch y salta a la secuencia de sentencias del case que hace match. Fijarse que la secuencia no tiene “llaves”..
- Se ejecuta las sentencias hasta que se encuentre un “**break**” (OJO: ¡¡¡el break es opcional!!!).
- Si ningún case coincide entonces se ejecuta la secuencia de sentencias asociadas con el default. La cláusula default es opcional. Eventualmente podría no ejecutarse nada.

Estructura de control condicional switch, ejemplo

Se lee un número 1, -1 o cero y se debe imprimir la expresión “Cero”, “Negativo”, “Positivo” o “Error” si no es 0,-1, 1.

```
switch ( n ) {  
    case 0:  
        System.out.println("Cero");  
        break;  
    case 1:  
        System.out.println("Positivo");  
        break;  
    case -1:  
        System.out.println("Negativo");  
        break;  
    default:  
        System.out.println("Error");  
}
```

switch avanzado (versión > 13)

Retornar directamente con un switch

```
System.out.printf(  
    switch(n) {  
        case 0 : yield "Cero";  
        case 1 : yield "Negativo";  
        case -1 : yield "Positivo";  
        default : yield "Error";  
    });
```

Múltiples etiquetas en el case

```
enum DIA {LUN,MIE,MAR,JUE,VIE,SAB,DOM};  
DIA d = DIA.valueOf(args[0]);  
  
System.out.printf(  
    switch(d) {  
        case LUN,MAR,MIE,JUE,VIE : yield "Entre  
semana";  
        default : yield "Fin de Semana";  
    });
```

switch avanzado (2) (versión ≥ 17)

Soporte de objetos y posibilidad de filtrar por expresiones

```
System.out.printf(  
    switch(obj) {  
        case String s -> "Cadena";  
        case Integer i when i > 0 -> "Positivo";  
        case Integer i when i == 0 -> "Cero";  
        default -> "Otro tipo";  
    });;
```

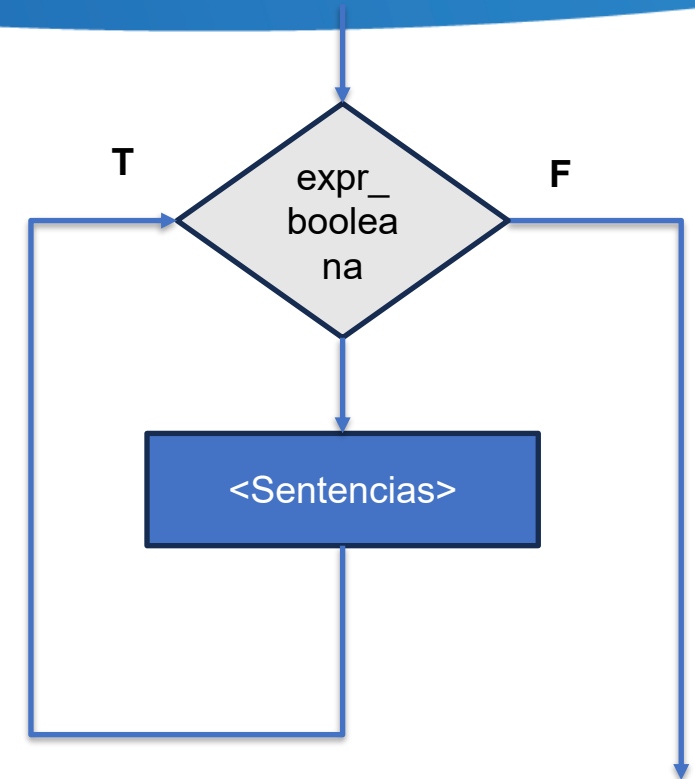

Estructuras de control

Repetición - while

```
while ( <expresion_booleana> )  
    <sentencia> | <bloque_sentencia>
```

- Características:

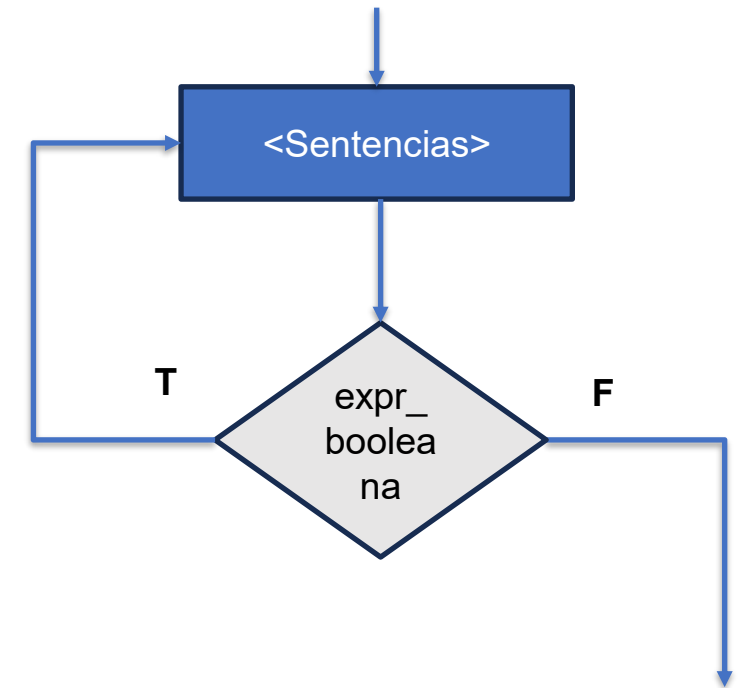
- Ciclo se ejecuta 0-N veces.
- Si la expresión booleana es true entonces se ejecuta la sentencia o el bloque de sentencias



Estructuras de control

Repetición - do-while

```
do {  
    <lista_sentencias>  
} while ( <expresion_booleana> );
```



- Características

- La lista de sentencias se ejecuta 1-N veces
- Mientras la expresión es true se mantiene en el ciclo

Estructuras de control

Repetición – do-while (Ejercicio)

Imprimir la cantidad de dígitos que tiene una variable entera n.

Aquí una posible solución en Java.

```
c = 0; //En c dejamos la cantidad de dígitos

do {
    n = n / 10; //n es el número
    c++;
} while ( n > 0 );

// Imprimir c
```

Estructuras de control

Repetición – for

```
for ( <expr_1>; <expr_2>; <expr_3> )  
    <sentencia> | <bloque_sentencia>
```

La semántica del for es:

```
<expr_1>  
while ( <expr_2> ) {  
    <sentencia> | <bloque_sentencia>  
    <expr_3>;  
}
```

for-each

Iterar por cada elemento del arreglo o de un objeto iterable. Coloca en <var> el elemento para que pueda ser utilizado en <bloque>.

```
for ( <tipo> var : <arreglo o iterable> ) {  
    <bloque>  
}
```



Parte IV

Cadenas y arreglos

- Cadenas (String)
- Arreglos

Cadenas

- Es un objeto, es decir, no es un tipo primitivo. Son dinámicos. Son inmutables.
- Es tratado por el compilador de una manera especial en relación a los otros “objetos” de otras clases. Por ejemplo, el tratamiento de la concatenación (+) y otras cosas (es inmutable.. *que es esto?*)
- Los caracteres componentes son Unicode no ASCII.
- Su tipo es **String**

```
String s = new String("Hola");
```

```
String s = "Hola"; // Preferible para constantes.
```

```
String s1 = "Hola"; // El compilador optimiza el espacio.
```

Cadenas (métodos usuales)

<code>char charAt(int index)</code>	Retorna el caracter en el índice index.
<code>String concat(String str)</code>	Concatena la cadena con str y retorna el resultado
<code>boolean endsWith(String str)</code>	Retorna true o false si la cadena contiene el sufijo str
<code>boolean equals(String str)</code>	Retorna true o false si la cadena es igual a str
<code>boolean equalsIgnoreCase(String str)</code>	Retorna true o false si la cadena es igual a str ignorando mayúsculas y minúsculas.
<code>int indexOf(char c)</code>	Retorna el índice de la primera ocurrencia del carácter c
<code>int indexOf(char c, int n)</code>	Retorna el índice de la primera ocurrencia del carácter c empezando en n
<code>int indexOf(String str)</code>	Retorna el índice de la primera ocurrencia de la subcadena str
<code>int indexOf(String str, int n)</code>	Retorna el índice de la primera ocurrencia de la subcadena str empezando en n
<code>int length()</code>	Retorna la cantidad de caracteres de la cadena
<code>String replace(char c1 ,char c2)</code>	Retorna la cadena resultante de reemplazar el caracter c1 por c2.
<code>String substring(int n1, int n2)</code>	Retorna la subcadena que empieza en n1 y termina en n2
<code>String toLowerCase()</code>	Retorna la cadena en minúsculas
<code>String toUpperCase()</code>	Retorna la cadena en mayúsculas
<code>String valueOf(obj)</code>	Retorna la cadena que representa el parámetro obj (boolean, int, float, double, etc)

Cadenas

Ejemplo: dada una cadena imprimir un “-” (guión) entre cada caracter de la misma.

Aquí una posible solución en Java.

```
String s = "HOLA";  
  
for ( int k=0; k < s.length(); k++ )  
    System.out.print(s.charAt(k) + "-");
```

¿Qué pasa cuando hago `<= s.length()`?

Arreglos

- ¿Qué es un arreglo? Dato estructurado cuyos componentes son del mismo “tipo”. Existen en todos los lenguajes y es una facilidad que se me brinda para solucionar más fácilmente algunas cuestiones.

- Ya lo vimos con el método main

```
public static void main (String [] arg)
```

- Declaración:

```
<tipo_dato> [] <identificador>
```

```
int [] A ; // Arreglo de ints.
```

```
String [] Palabras; // Arreglo de Cadenas.
```

Arreglos

- Un arreglo es en realidad un “objeto” para Java. Es un tipo de dato “referencia” (*un puntero*)
- Cuando yo declaro un arreglo fijarse que no le digo la cantidad de elementos que contendrá (como en C o C++).
- Instanciación (en Java quiere decir “creación”)

```
<var_Arreglo> = new <tipo_dato_arreglo> [<tamaño>]
```

```
int [] A;
```

```
A = new int [100] ; // A ahora tiene 100 ints.
```

- La variable de tipo arreglo declarada pero no instanciada contiene tiene por defecto el valor “*null*”, que es como decir “*no apunta a nada por ahora*”.

Arreglos

- Acceso a componentes:

```
<var_Arreglo>[<indice>]
```

```
int [] A = new int[100] ; //podemos inicializar.
```

```
A [0] = 100 ; //fijarse que el 1er. elemento esta en el índice 0.
```

- Instanciación (Inicialización estática)

```
int [] A = {10,20,30,40,50};
```

```
String [] Palabras = {"Hola", "que", "tal"};
```

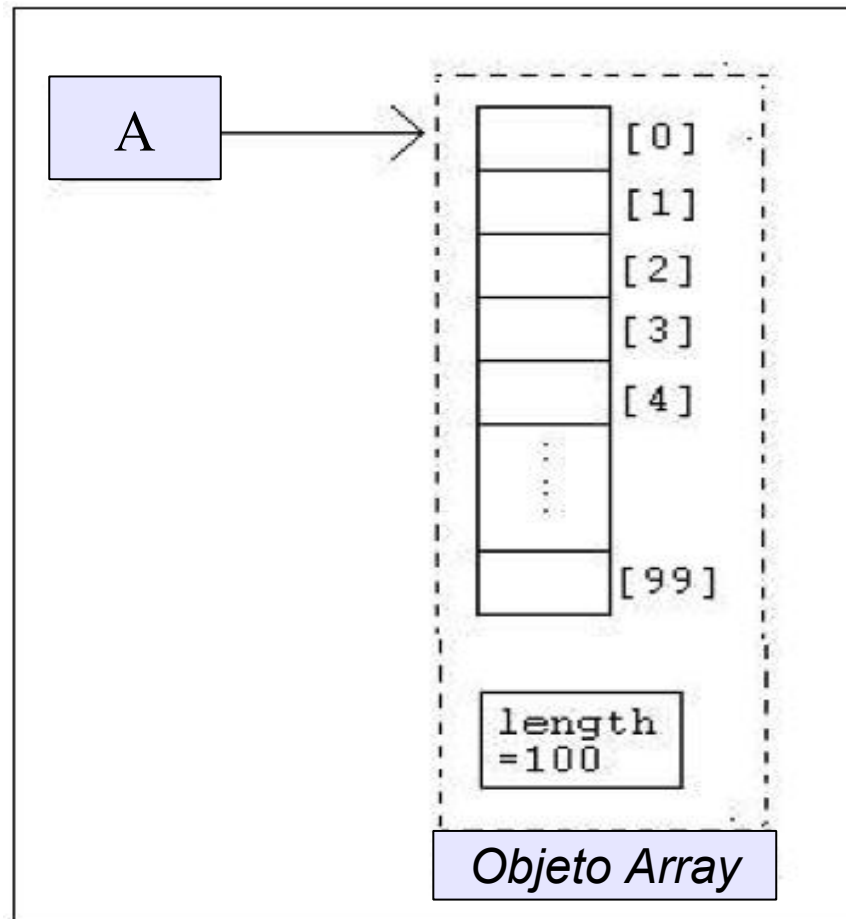
```
boolean [] Resultados = {true, false, true, true};
```

- Cantidad de elementos:

```
<var_Arreglo>.length //Fijarse que no tiene ()
```

Arreglos

Ejemplo: `int [] A = new int[100];`



Arreglos multidimensionales

```
int [] [] A = { { 10,20,30,40,50} ,  
                { 10,20,30,40,50}  
              };
```

```
System.out.println(A[0][0]); // Imprimiría 10
```

```
String [] [] Palabras = { {"Hola"}, {"que"}, {"tal"} };
```

```
// imprimir todo el contenido de Palabras.
```

```
for (int f=0; f < Palabras.length; f++ )
```

```
    for ( int c=0; c < Palabras[f].length; c++ )
```

```
        System.out.println(Palabras[f][c]);
```

Arreglos con for-each

//Ejemplo 1

```
String [] frutas = {"manzana", "banana", "pera", "naranja"};
for ( String fruta : frutas )
    System.out.println(fruta);
```

Pruebe en
JSHELL

//Ejemplo 2

```
int [] edades = {20, 21, 39, 55, 19, 29, 29};
int edad = -1; //asumir que no se tendrá edad negativa.
for ( int edad : edades)
    if ( edad > mayor )
        mayor = edad;

if ( mayor >= 0 ) System.out.printf("Mayor edad es %d", edad );
else                System.out.println("Existe un error de dato");
```

Parte V

Introducción a POO. Clases

- Conceptos básicos sobre Programación Orientada a Objetos.
- Clases. Definición. Uso. Características en Java.
- Ejercicios sencillos de definición de clases y de uso.

Repaso

- Cadenas. Recordar que es un tipo de dato que en Java es inmutable.
 - Recordar la diferencia entre utilizar new y asignación directa con el operador = .
- Arreglos.
 - Es un tipo de dato estructurado de tamaño dinámico.
 - Es mutable. Para Java es un tipo de Objeto particular.
 - El primer elemento siempre empieza en la posición cero.
 - Se “instancia” (recordar que es esto) con el operador new

Conceptos básicos sobre POO

- Recordar que la OO es una técnica cuyo objetivo es de alguna forma “modelar” el mundo real de una forma más natural.
- Detrás de esto existe una serie de ventajas:
 - Escribir menos, mejorar el mantenimiento del código, aprovechar el código ya escrito entre otras cosas.
- Existen tres características básicas para que un lenguaje sea considerado orientado a objetos:
 - Encapsulamiento de datos (asociar dato y proceso aplicables a los datos). Ocultamiento de información.
 - Herencia.
 - Enlace dinámico (polimorfismo).

Conceptos básicos

- **Clase:** se puede decir que es la definición de las características de cierto tipo de objeto del mundo real. Por ejemplo, *Persona* es una clase. *Auto* es una clase. En un programa Java es una estructura sintáctica que permite definir datos y operaciones (es una implementación de TAD). Se puede ver como una plantilla de datos.
- **Objeto:** es una instancia “viva” de cierta clase.
 - *Persona* es la clase y *Juan Perez* es una instancia de la clase *Persona*.
- Las clases tienen dos componentes fundamentales:
 - **Atributos** (características del objeto) o datos miembros
 - En *Persona* sería: apellido, nombre, estatura, peso, etc.
 - **Métodos** (qué puedo hacer o que puede hacer el objeto).
 - En *Persona* sería: *cambiarPeso()*, *cambiarApellido()*, etc.

Sintaxis de creación de clases

Para definir un clase en Java

```
<modificador> class <identificador_clase> {  
    <atributos>*  
    <constructores>*  
    <métodos>*  
}
```

donde

- <modificador> indica es el modificador de acceso a la clase (public, private, protected, package)

Ejemplo: *Persona*

- Qué características comunes tienen las Personas (atributos)
- Qué acciones pueden hacer las personas o pueden aplicarse a las Personas. (métodos aplicables).
 - Persona:
 - apellido, nombre, cedula, fecha de nacimiento, etc.
 - Qué podemos hacer
 - imprimir sus datos, asignar su nuevo nombre, cambiar su cédula, corregir su nombre, etc, etc.

Persona. Definición Java

```
public class Persona {  
    private String nombre ;  
    private String apellido ;  
    private String cedula;  
  
    /* Constructor de persona */  
    Persona ( String nom, String ap, String ced ) {  
        this.nombre      = nom;  
        this.apellido    = ap;  
        this.cedula      = ced;  
    }  
  
    public void imprimirDatos(){  
        System.out.println("Cedula   : " + this.cedula   +  
                           "Apellido: " + this.apellido +  
                           "Nombre   : " + this.nombre );  
    }  
    .....  
}
```

Usando la clase Persona

```
import Persona; // No es necesario si estoy en el mismo directorio

public class UsoPersona {

    public static void main (String [] args ) {

        Persona per1 = new Persona ("1", "Juan", "Perez");
        Persona per2 = new Persona ("2", "Maria", "Lopez");

        per1.imprimirDatos();

        per2.imprimirDatos();

    }

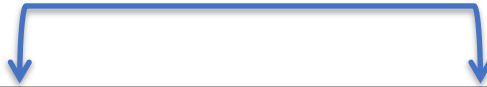
}
```



Variables de instancia vs. Variables de clase

- Las **variables** de instancia se crean por cada instancia (cada nuevo objeto creado con el operador **new**).
- Las **variables** de clase se crean una vez por clase.

instancias



Clase Auto		Objeto Auto A	Objeto Auto B
Variables de instancia	Número de placa	ABC 111	XYZ 123
	Color	Azul	Rojo
	Fabricante	Nissan	Toyota
	Velocidad máxima	120 km/h	180 km/h
Variable de clase	Cantidad = 2		
Métodos de instancia	Método Acelerar		
	Método Encender		
	Método Frenar		

Variables de instancia

No tienen el modificador “*static*” por delante.

```
public class Auto {  
    private int color;  
  
    private int cantPuertas;  
  
    private String Marca;  
  
    private String Modelo;  
  
    . . . . .
```

- Estas variables existen por cada instancia creada de la clase (por cada **new** que hago)

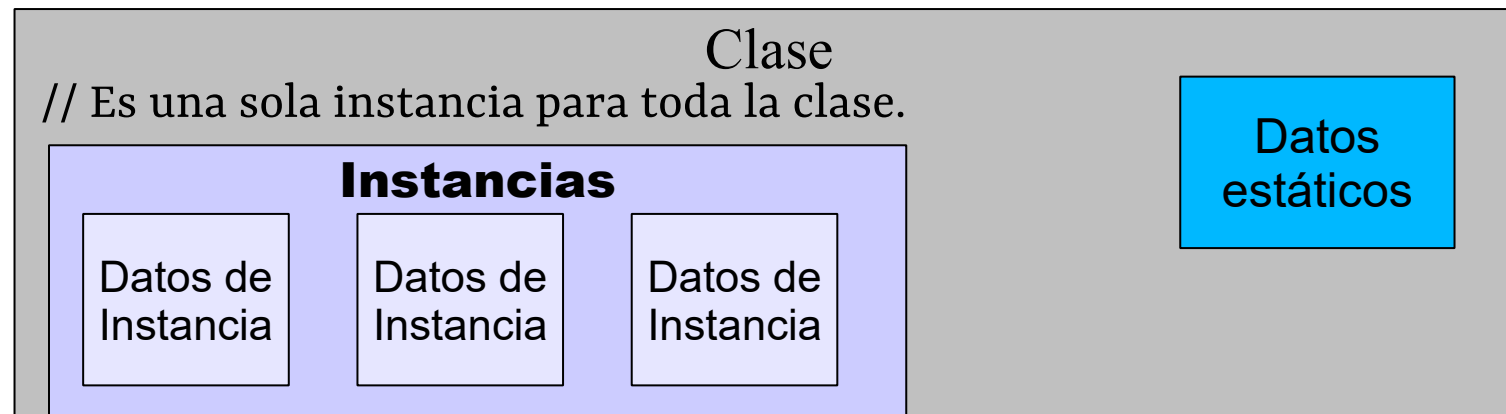
Variables de clase

- Tienen el modificador “static” por delante.

```
public class Auto {
```

```
    private static int cantidad_Autos_Definidos;
```

```
    . . . . .
```



Métodos de instancia vs. Métodos estáticos

- **Métodos de instancia:** actúan sobre variables de instancia o variables de clase. No tienen “static” como modificador.
- **Métodos de clase:** actúan solo sobre variables de clase y pueden ser invocados sin necesidad de instanciar ningún objeto.

Ejemplos:

```
System.out.println(...) // Mirar en la documentación la clase
                        // System y el dato miembro out.
Integer.parseInt(..)    // Mirar la documentación.
```

Accesos a variables y métodos

- **public**
 - Cualquier clase puede acceder.
- **package** o **friendly** (cuando no se define modificador este es el acceso otorgado)
 - Pueden acceder clases del mismo paquete (y la misma clase)
- **protected**
 - Pueden acceder solo clases “hijas” (y la misma clase)
- **private**
 - Solo la propia clase puede acceder

Constructores

- Es un método especial que tiene el mismo nombre de la clase.
- Su objetivo es ejecutar código de inicialización sobre el objeto que se está creando.
- Se le llama justo en el momento de instanciar al objeto.
- Si no colocamos ningún constructor Java coloca uno por defecto.
- Pueden existir muchos constructores que se diferencian por la cantidad y tipo de los parámetros que recibe.

Constructores - ejemplo (definición)

```
public class Persona {
```

```
...
```

```
Persona ( ) {  
    apellido = "";  
    nombre   = "";  
    cedula   = "";  
}
```

```
Persona (String a, String n, String c) {  
    apellido = a;  
    nombre   = n;  
    cedula   = c;  
}
```

```
.....
```

Constructores - ejemplo (llamada)

```
import Persona;

public class UsoPersona {

    .....

    public static void main (String [] args ) {

        Persona per1 = new Persona();
        Persona per2 = new Persona ("1", "Juan", "Perez");

        .....
    }
}
```

Ámbito de variables

- Si son declaradas en la clase es la clase entera (pueden declararse en cualquier parte de la clase, pero usualmente se colocan al principio). Se denominan *datos de clase*. En su uso pueden tener delante *this* que significa que hace referencia a la instancia actual.
- Si son declaradas en el método, solo visibles el método. Se denominan *variables locales*.
- Si son declaradas en el bloque, solo en el bloque.

```
public class Test {
```

```
    private int j = 0;
```

Dato de clase (Instancia o de clase)

```
Test ( int k ) {
```

```
    this.i = k;
```

this se refiere a la instancia actual

```
}
```

```
public static void main (String [] args ) {
```

Variables locales

```
    int i = 1;
```

```
    int j = 0;
```

```
    while ( i > j ) {
```

```
        int k = i
```

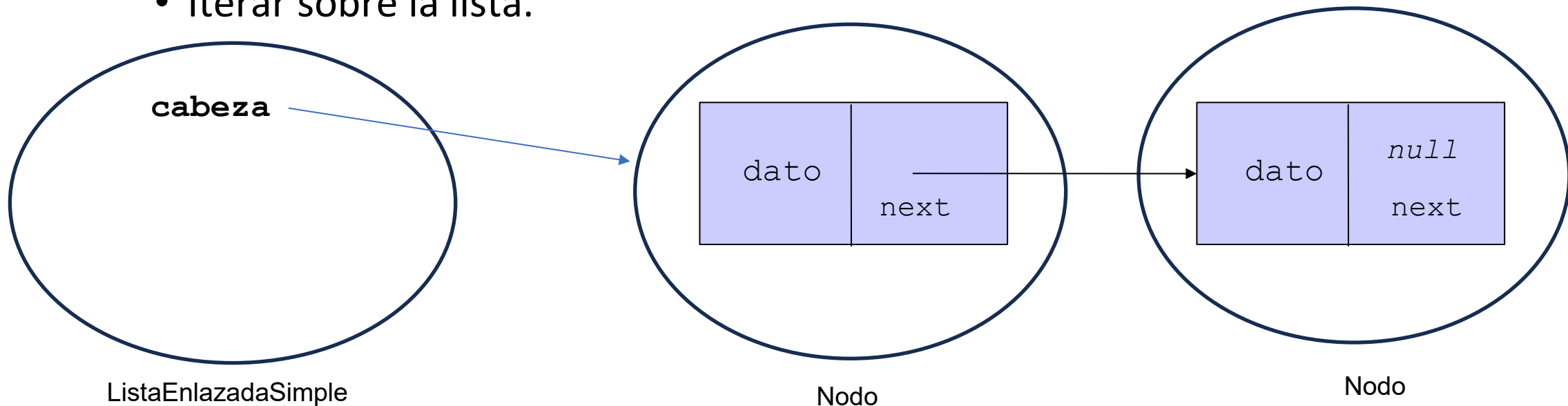
Variable local de bloque

```
        ..
```

```
    }
```

```
}
```

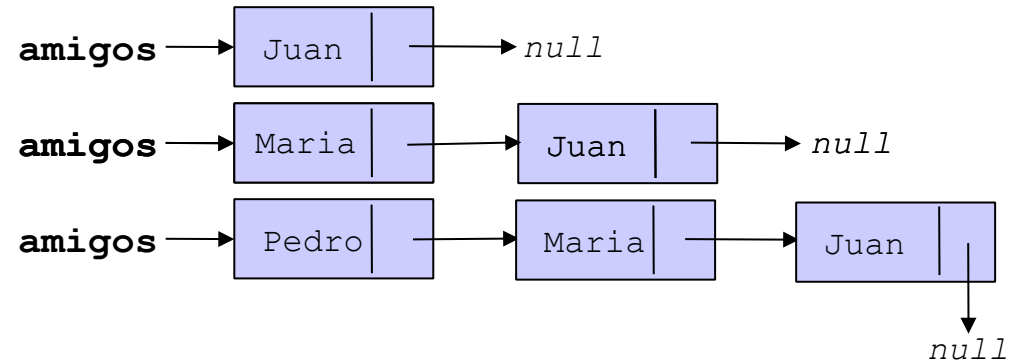

- Defina la clase ***ListaEnlazadaSimple*** con las siguientes operaciones:
 - Agregar (por el frente de la lista)
 - Remover (del frente de la lista)
 - Iterar sobre la lista.



Ejemplo de uso de *ListaEnlazadaSimple*

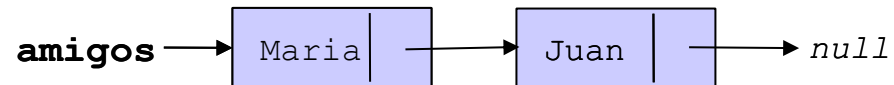
```
ListaEnlazadaSimple amigos = new ListaEnlazadaSimple();
```

```
amigos.agregar("Juan");  
amigos.agregar("Maria");  
amigos.agregar("Pedro");
```



```
amigos.imprimir();
```

```
// Borramos el primero de la lista  
amigos.remove();
```



```

public class ListaEnlazadaSimple {

    private Nodo cabeza;

    ListaEnlazadaSimple() {
        cabeza = null;
    }

    void agregar (String d)
    {
        Nodo tmp = cabeza;
        cabeza = new Nodo(d);
        cabeza.setNext(tmp);
    }

    void imprimir() {
        Nodo actual = this.cabeza;
        while ( actual != null ) {
            System.out.print(actual.getDato());
            actual = actual.getNext();
        }
    }
}

```

```

class Nodo {
    private String dato;
    private Nodo next;

    Nodo (int d ){
        this.dato = d;
        this.next = null;
    }

    public int getDato() {
        return this.dato;
    }

    public Nodo getNext() {
        return this.next;
    }

    public void setNext( Nodo n)
    {
        if ( n != this )
            this.next = n;
        else
            throw new IllegalArgumentException ("No
            se puede apuntar a si mismo!!");
    }
}

```

Escriba este código.

Luego acceda a Jshell y compile la clase del archivo ListaEnlazadaSimple.java

Agregue 100 elementos de forma aleatoria



Parte VI

Herencia

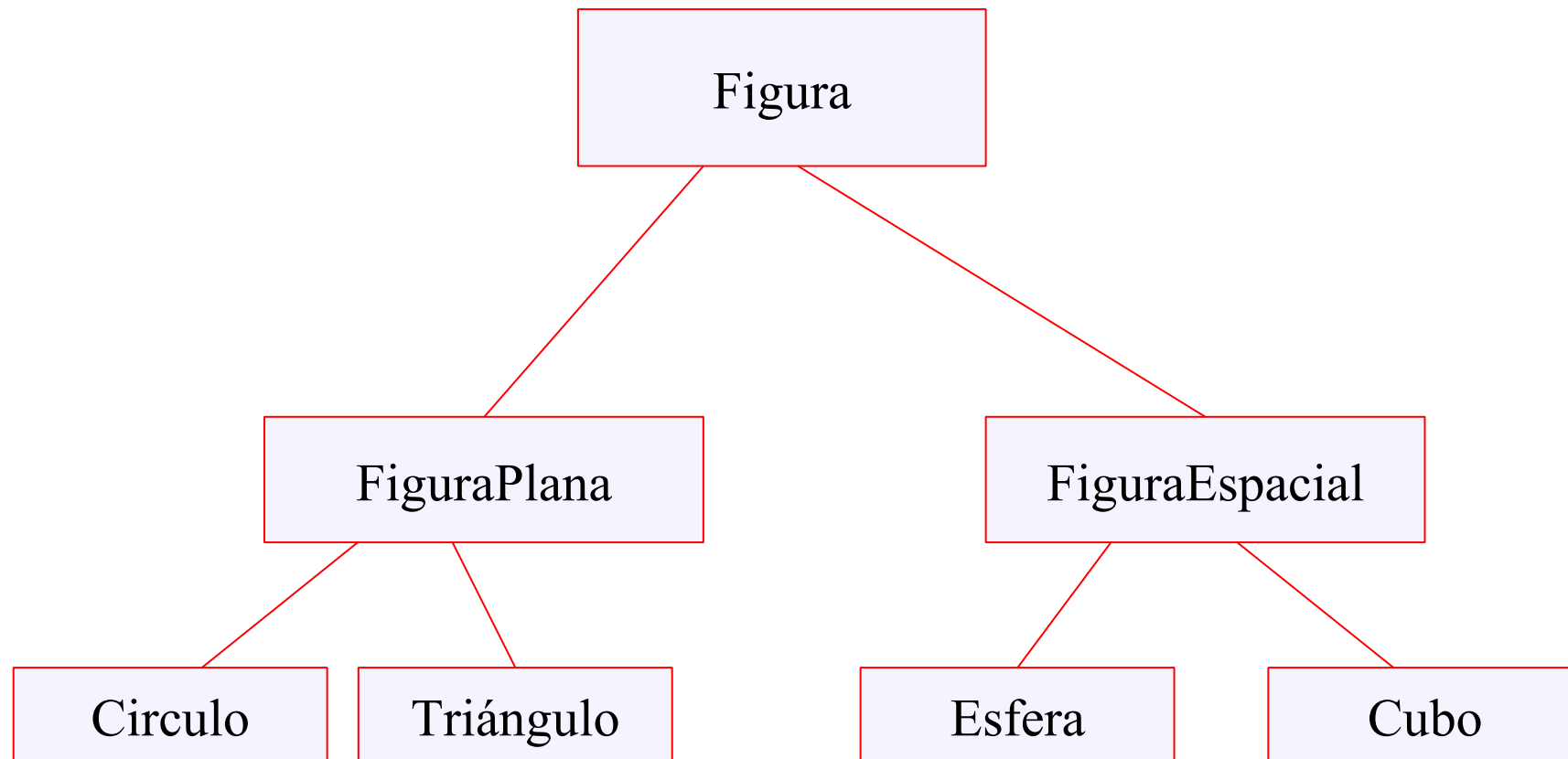
- Herencia. Conceptos básicos y forma de aplicarlo en Java.
- Ejercicios sencillos de definición de clases y sub-clases.

Herencia (usaremos poco en el curso)

- **Objetivo principal:** reutilización de código
 - Uno define características generales en una clase y se heredan en las clases hijas
 - Las clases hijas heredan todas las propiedades y métodos, pueden definir sus propios atributos y métodos o cambiar funcionalidades ya existentes.
- **Definiciones**
 - superclases
 - subclases o clases hijas

Ejemplo

fijarse que se forma una jerarquía



Herencia en Java

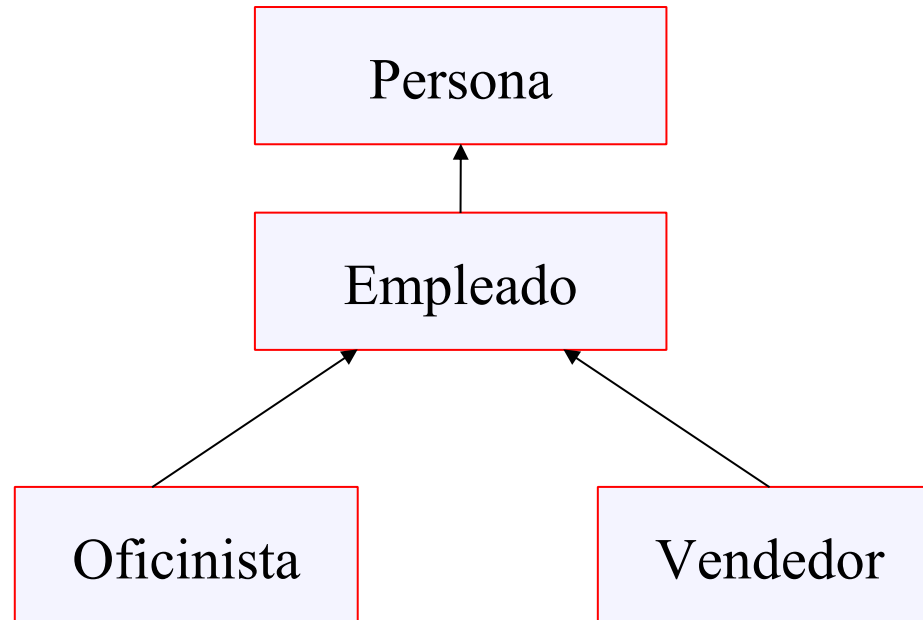
```
public class Figura {  
    private String nombre;  
    public float dibujar ();  
}  
  
public class FiguraPlana extends Figura{  
    private int x; // Coordenada  
    private int y; //  
  
    public float superficie();  
}  
  
public class Circulo extends FiguraPlana {  
    private float diametro;  
    public float superficie () { float radio = diametro/2;  
        return Math.PI*radio*radio;  
    }  
    ...
```

Algunas cuestiones de Java

- Todos los métodos son virtuales (modificables en las clases hijas) excepto cuando se coloca el modificador *“final”*.
- Solo existe Herencia simple (en C++ hay herencia múltiple). (Una clase puede tener un solo “padre”)
- Cualquier clase en Java siempre es hija de la clase Object (esta clase tiene algunas funcionalidades ya definidas).
- Los constructores no se heredan.

Ejemplo de Herencia

- **Extender la clase Persona:** de forma que ahora tengamos tres clases derivadas. Empleado y este tenga dos subclases que sea Oficinista y Vendedor.



Polimorfismo y enlace dinámico

- La herencia permite “heredar/tomar” las características de una superclase agregando características particulares. Una subclase o clase hija es una especialización de la superclase.
Una instancia de la clase hija es también una instancia de la superclase pero no de manera inversa.
- Por ejemplo, un *Vendedor* es una *Persona* pero una *Persona* no es un *Vendedor* precisamente (en el ejemplo anterior).
`Persona p = new Vendedor();` ✓
`Vendedor v = new Persona();` ✗
- **Polimorfismo** es la capacidad de que un objeto de un supertipo o superclase puede referir a un objeto de un subtipo o subclase.
- **Enlace dinámico** es la característica del sistema (JVM) de llamar a la versión adecuada del método de acuerdo a la instancia del objeto.

Polimorfismo y enlace dinámico ejemplo

```
public class Demo {  
    public static void main (String [] args ) {  
        Estudiante e = new Estudiante(..);  
        Docente d = new Docente (..);  
  
        imprimir_dato(e);  
        imprimir_dato(d);  
        ..  
    }  
  
    public static void imprimir_dato (Persona p)  
    {  
        System.out.println(p.datos());  
    }  
}
```

Polimorfismo, puedo recibir en p variables de tipo Persona o de clases hijas de Persona

Enlace dinámico, se ejecutará el método datos() de acuerdo a la instancia (de Estudiante o de Docente)

```
public class Persona {  
    ..  
    public String datos () {  
        ..  
    }  
}  
  
public class Estudiante extends Persona  
{  
    ..  
    public String datos () {  
        ..  
    }  
}  
  
public class Docente extends Persona {  
    ..  
    public String datos () {  
        ..  
    }  
}
```



Parte VII

Manejo de Excepciones

- Excepciones. Concepto. Definición y uso de excepciones.
- Agregar excepciones a la jerarquía de clases implementada.

Manejo de Excepciones

- ¿Qué es una excepción?
 - Evento inusual que ocurre en tiempo de ejecución y que causa una interrupción en el flujo normal de ejecución
 - Ejemplos:
 - División por cero
 - Acceso fuera de los límites de un arreglo
 - Entrada inválida
 - Problemas en el disco
 - Archivo no existe
 - No hay más memoria
 - No hay conexión de red
 - Etc, etc

Ejemplo de una excepción

Pruebe en
JSHELL

```
1. public class DivisionPorCero {  
2.     public static void main(String args[]) {  
3.         System.out.println(3/0);  
4.         System.out.println("Luego de la division.");  
5.     }  
6. }  
7. // Como reacciona este programa?
```

Excepción – Salida del ejemplo

```
Exception in thread "main" java.lang.ArithmeticException: / by zero  
    at DivisionPorZero.main(DivisionPorCero.java:3)
```

- En Java el manejador de excepciones por defecto:
 - Es proveído por el runtime (JRE)
 - Imprime la descripción de la excepción.
 - Imprime la llamada de los métodos que causó la excepción
 - Termina el programa

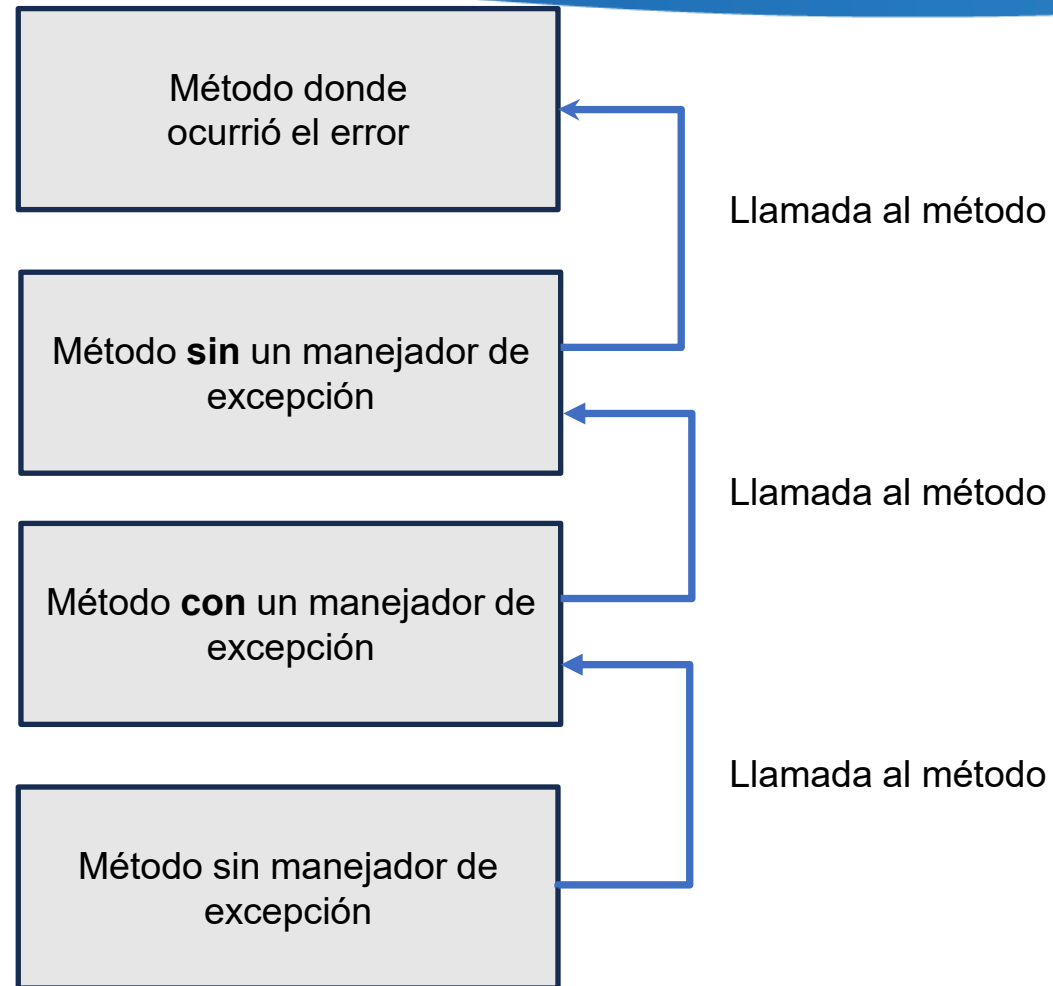
Excepciones

¿Qué ocurre cuando existe una excepción en un método?

- Se crea un “objeto” de tipo *Exception* que es pasado al intérprete. Esto se denomina “*lanzamiento de una excepción*” (**throw**).
- El objeto de excepción contiene información acerca del error que ocurrió.
- El Runtime busca en la secuencia de llamadas el método que manejaría la excepción, finalmente si no lo encuentra el programa termina.

Excepciones

Ejemplo de funcionamiento



Excepciones

- Si existe un método que pueda manejar la excepción
 - El Runtime pasa el objeto de Excepción a este método.
 - El método en este caso define las excepciones que se atraparán. (**catch**)

Excepciones

Ejemplo de funcionamiento

Se genera la excepción

Método donde
ocurrió el error

La excepción de propaga

Método sin manejador de
excepción

La excepción se atrapa

Método sin manejador de
excepción

Método sin manejador de
excepción

Búsqueda por el manejador
apropiado

Búsqueda por el manejador
apropiado

Excepciones - Beneficios

- No mezclamos el manejo de errores con el código “normal”.
- Los errores pueden propagarse
- Podemos agrupar o categorizar los tipos errores (ya que las excepciones son objetos de la clase Exception)

Excepciones

Atrapando excepciones

```
try {  
    <codigo_protegido>  
} catch (<TipoExcepcion> <Variable>) {  
    <Manejador de la excepción>  
}  
  
...  
  
} catch (<TipoExcepcion> <Variable>) {  
    <Manejador de la excepción>  
}
```

Excepciones – try-catch (ejemplo)

```
1 public class DivisionByCero {
2     public static void main(String args[]) {
3         try {
4             System.out.println(3/0);
5             System.out.println("Luego de la division.");
6         } catch (ArithmeticException exc) {
7             //Division por zero es ArithmeticException
8             System.out.println(exc);
9         }
10        System.out.println("Luego de la excepción.");
11    }
12 }
```

Pruebe en
JSHELL

Excepciones – múltiples catch

```
1 public class MultipleCatch {  
1     public static void main(String args[]) {  
2         try {  
3             int den = Integer.parseInt(args[0]);  
4             System.out.println(3/den);  
5         } catch (ArithmeticException exc) {  
6             System.out.println("Divisor fue 0.");  
7         } catch (ArrayIndexOutOfBoundsException exc2) {  
8             System.out.println("Faltan argumentos");  
9         }  
10        System.out.println("Luego de la excepción.");  
11    }  
12 }
```

Pruebe en
JSHELL

Excepciones - finally

```
try {  
    <codigo_protegido>  
  
} catch (<TipoExcepcion> <Identificador>) {  
  
    <Manejador_de_la_excepcion>  
  
} ...  
  
} finally {  
    <finalización>  
  
}
```

Código se ejecuta antes
de que el bloque
try-catch termine
(exista o no excepción)

Lanzar excepciones

- Un método siempre debe atrapar las excepciones o listar las que va a dejar pasar. Excepto para las clases Error o RuntimeException o sus derivados.
- Si un método puede causar una excepción y no la atrapa entonces debe mencionar en su definición que tipo de excepciones dejará pasar (usando la cláusula throws)

```
<tipo> <Nombre_Metodo> (<lista_parametros>) throws <Lista_Excepciones_no_atrapadas>
{
    <cuerpo_método>
}
```

Ejemplo de uso de throws

```
import java.io.*;

public class TestExcepcion {

    public static int leer_int ( ) {
        BufferedReader stdin = new BufferedReader( new InputStreamReader (System.in));

        String linea = "";
        int n = 0;

        linea = stdin.readLine();
        n = Integer.parseInt(linea);
        return n;
    }

    public static void main( String[] args) {

        int n = leer_int();

        System.out.println("El valor leido fue " + n);

    }

}
```

```
D:\ccappo\Dropbox\2024-Alg3-2S\java>javac TestExcepcion.java
TestExcepcion.java:9: error: unreported exception IOException; must be
caught or declared to be thrown
        linea = stdin.readLine();
                        ^
1 error
```

Uso de throws

```
public static int leer_int ( ) throws Exception {
    BufferedReader stdin = new BufferedReader( new InputStreamReader (System.in));

    String linea = "";
    int n = 0;

    linea = stdin.readLine();
    n = Integer.parseInt(linea);
    return n;
}

public static void main(String[] args) throws Exception {
    int n = leer_int();

    System.out.println("El valor leído fue " + n);
}
```

Mejor con Scanner

```
public static int leer_int ( )    {
    Scanner sc = new Scanner (System.in);

    int n = 0;

    while ( true ) {
        try {
            n = sc.nextInt();
            break ;
        }
        catch ( InputMismatchException e ) {

            System.out.print("Valor inválido!, intente de nuevo")
            sc.next(); //consumimos la cadena inválida
        }
    }

    sc.close();

    return n;
}
```

Notar aquí que la excepción lanzada por Scanner cuando lo leído de la entrada estándar no puede ser interpretado al tipo indicado entonces se genera una excepción tipo **no chequeada** (se verá más adelante) denominada *java.util.InputMismatchException*

Utilizamos la generación de excepción para darnos cuenta del error de tipeo y volver a solicitar el usuario.

Pruebe en
JSHELL

Creando nuestro propio tipo de excepción

- Por ejemplo, en el caso de Fecha creamos una excepción en caso de que los parámetros de año, mes y día son incorrectos.
- Asumir que el año debe estar entre 900 y 2099, el mes debe estar en 1 y 12 y el día debe estar entre 1 y 31.
 - La cantidad máxima para cada mes (de enero a diciembre) es:
31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31.
 - Para Febrero es 29 el tope si el año es bisiesto. Un año es bisiesto si es múltiplo de 4 pero no de 100 (a no ser que sea de 400).

Ejemplo de definición de Excepción

```
class FechaInvalida extends Exception {  
    FechaInvalida ( String msg ) {  
        super(msg);  
    }  
}  
  
public class Fecha {  
    public Fecha ( int a, m , d ) throws FechaInvalida {  
        if ( esFechaValida(a,m,d)) {  
            throw new FechaInvalida("La fecha es incorrecta");  
        }  
    }  
    ....  
}
```

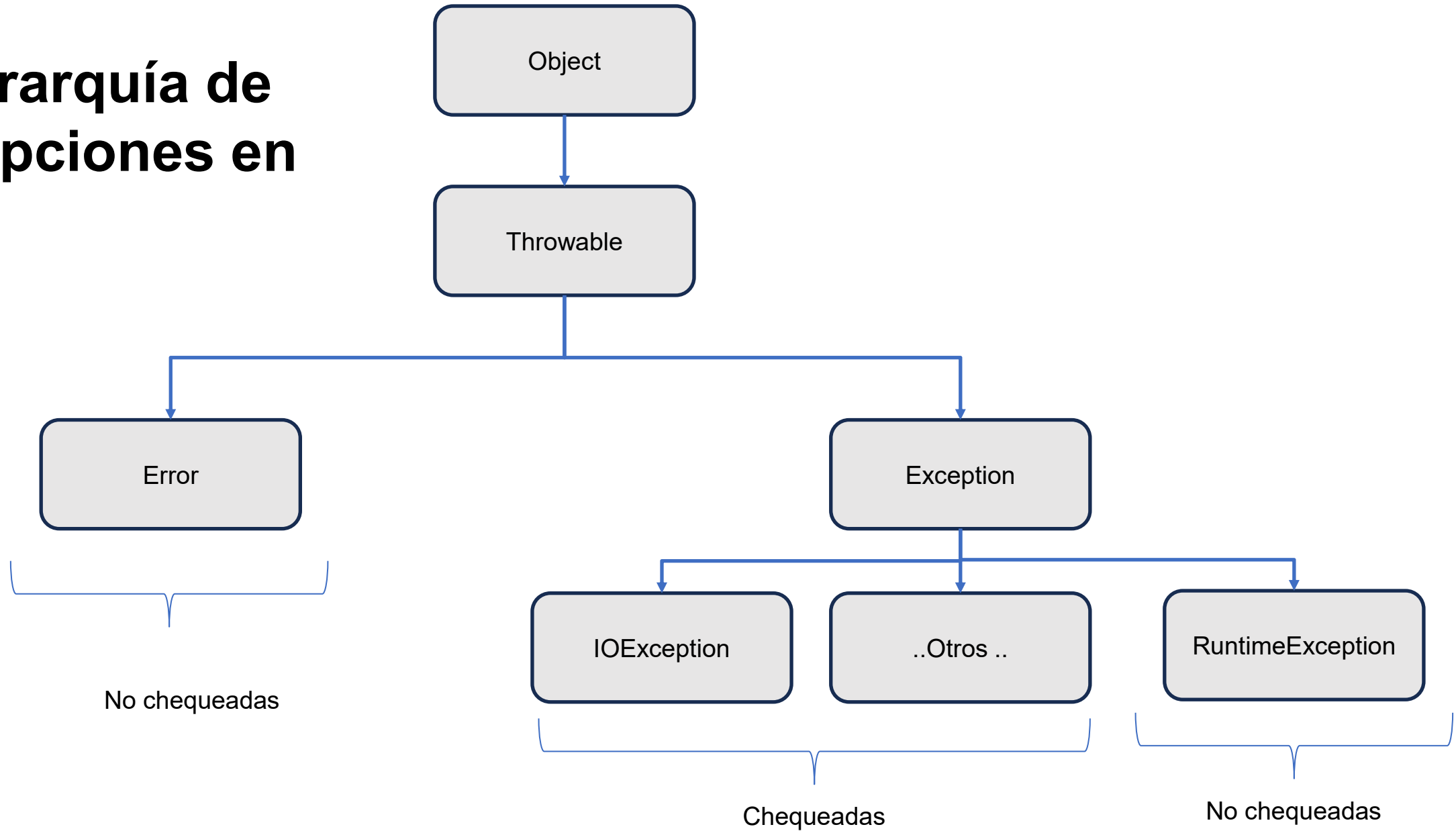
La jerarquía de Excepciones en Java

- Throwable
 - Es la raíz de todas las excepciones
 - Dos subclases
 - Error : generadas por el propio runtime,
 - Exception : las que pueden ser “controladas” por el programador.

A tener en cuenta

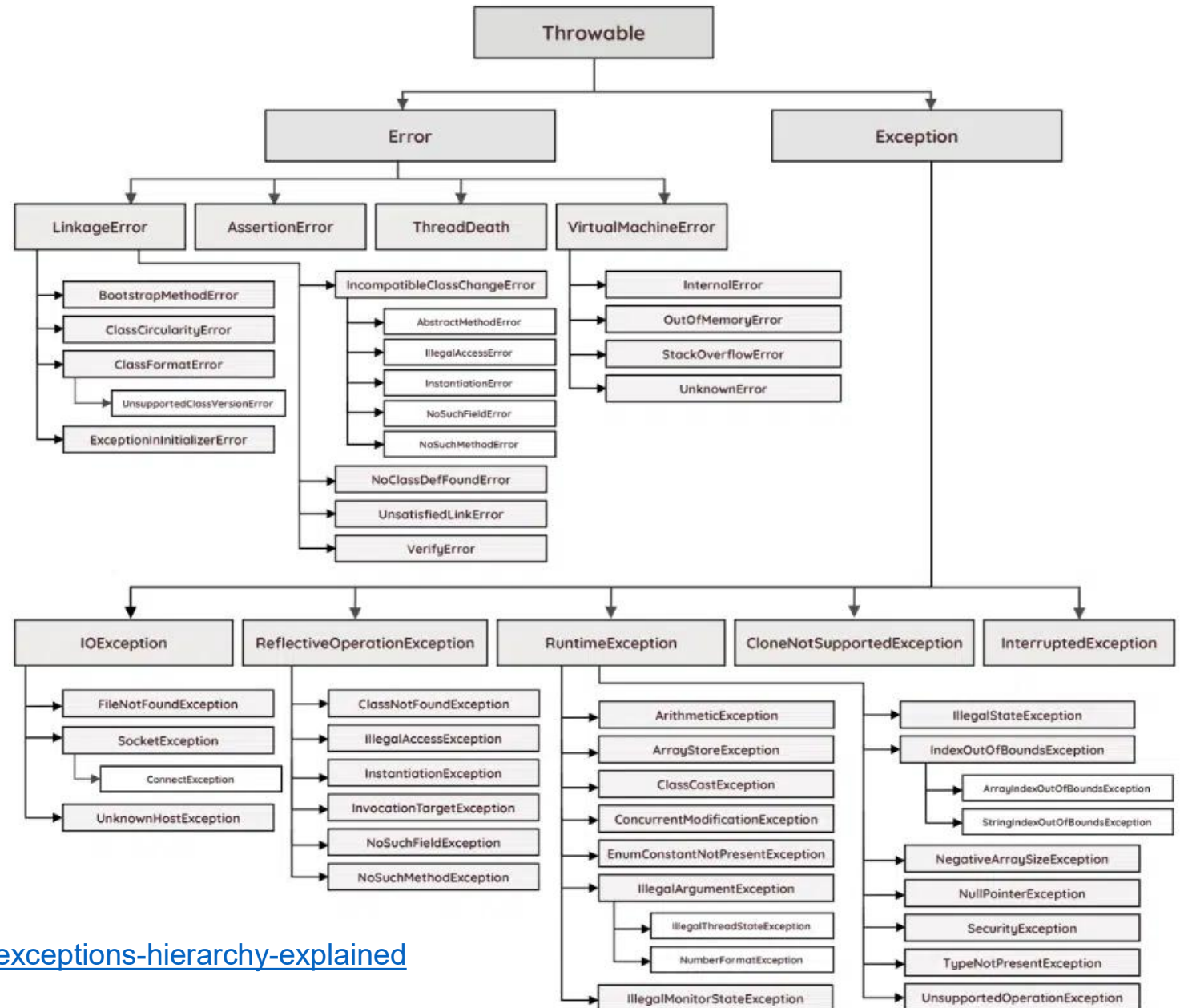
- Existen dos tipos de excepciones:
 - Checked (comprobadas):
 - El compilador detecta que sean atrapadas o dejadas pasar.
 - Derivan de *Exception* excepto `RuntimeException`
 - Unchecked (no comprobadas)
 - Se refieren a excepciones que no se detectan en tiempo de compilación
 - `Error`, `RuntimeException` (por ejemplo, `ArithmeticException`).

La jerarquía de Excepciones en Java



La jerarquía de Excepciones en Java

Ejemplo de clases específicas de excepciones



Tomado de <https://rollbar.com/blog/java-exceptions-hierarchy-explained>

Parte VIII

Clases abstractas, interfaces y generic

- Clases Abstractas. Uso y definición en Java.
- Interface. Concepto. Uso y definición en Java. Ejemplo sencillo de uso.
- Generics. Concepto. Utilización. Sintaxis y ejemplos sencillos.

Clases abstractas

- Una clase abstracta es que aquella que tiene uno o más métodos abstractos, un método es abstracto si no tiene implementación o cuerpo
public abstract float getSuelo();
- Una clase abstracta también puede ser definida como:
public abstract class Empleado {..};
- Estas clases no pueden ser instanciadas.
- Sus descendientes tienen que implementar los métodos abstractos.
- Permiten “definir” funcionalidades básicas que deben tener las clases hijas, sin decir todavía “como”.

Clases abstractas

- Un ejemplo de esto puede aplicarse a la clase Empleado.
- En la empresa no existe un Empleado que no sea Oficinista, Vendedor u otro. Pero necesitamos definir las funcionalidades generales de Empleado para tener métodos que actúen sobre cualquier tipo de Empleado.
- Ejemplos:
 - Cambiar en la clase Empleado los métodos de getSuelo y getTipoEmpleado añadiéndoles el modificador abstract
 - Qué pasaría si intentamos instanciar la clase Empleado?
Empleado e = new Empleado("1", "Torres", "Juan");

Interfaces

- Es una forma de definir el comportamiento de las clases, es lo que se denomina “software por contrato”.
- Es como una clase abstracta pero sin ninguna implementación.
- Se pueden definir en una interfaz métodos y constantes.
- Todos los métodos son abstractos, es decir, sin implementación
- Una clase concreta puede “implementar” una interface, lo que le obliga concretar los métodos definidos en la interfaz.
- Una clase puede implementar más de una interface.

Interface

Ejemplo de definición

```
public interface Serial {  
    int siguiente();  
    void reset();  
    void reiniciar(int x);  
}
```

```
public class PorDos implements Serial {  
    private int actual;  
    private int inicio;
```

```
    public PorDos() {  
        reset();
```

```
    0;
```

```
    }
```

```
        public int siguiente() {  
            actual += 2;  
            return actual;  
        }
```

```
    } /* Fin de la clase PorDos */
```

```
    public void reset() {  
        actual = 0;
```

```
        inicio =
```

```
    }
```

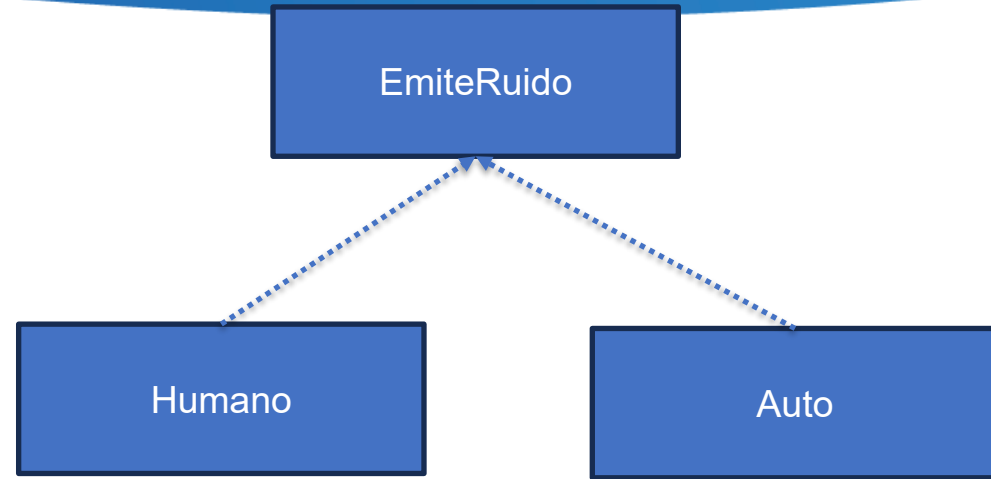
```
    public void reiniciar(int x) {  
        actual = x;  
        inicio = x;  
    }
```

Interface - ejemplo 2

```
public interface EmiteRuido {  
    public void hacerRuido();  
}
```

```
public class Humano implements EmiteRuido {  
    private String nombre;  
  
    public Humano (String s) { this.nombre = s};  
  
    public void hacerRuido() {  
        System.out.println("HOLA, yo puedo hablar y soy " + nombre);  
    }  
}
```

```
public class Auto implements EmiteRuido {  
    public void hacerRuido() {  
        System.out.println("RRRRRRRRMMMMMM");  
    }  
}
```



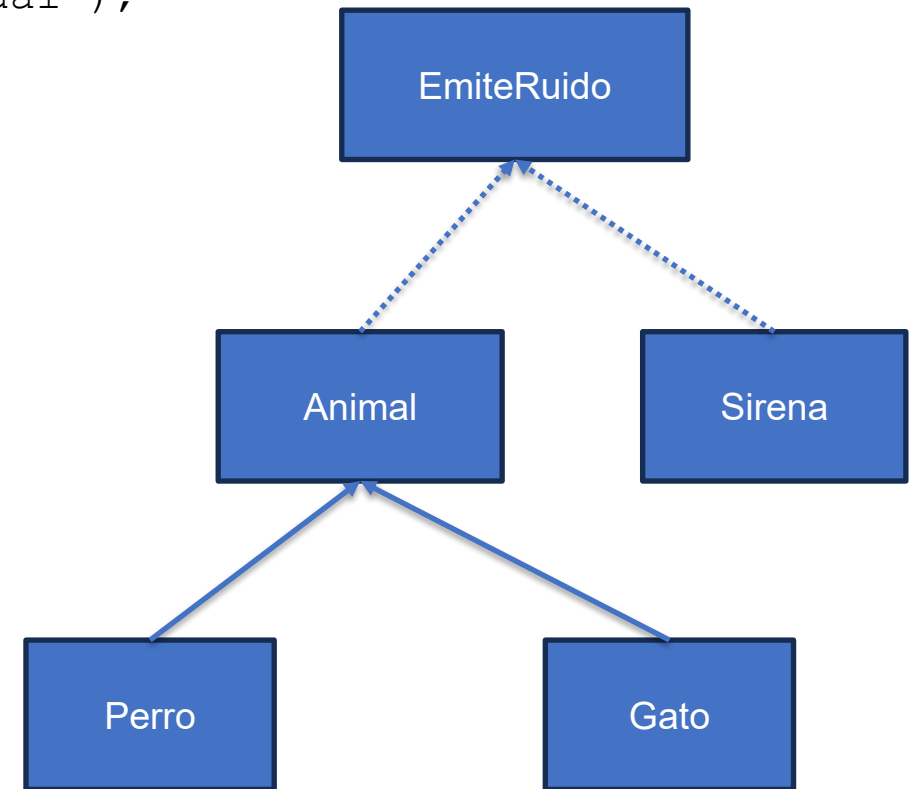
Interface

```
public class Animal implements EmiteRuido {  
    public void hacerRuido() {  
        System.out.println("Soy un animal pero no se cual");  
    }  
}
```

```
public class Perro extends Animal {  
    public void hacerRuido() {  
        System.out.println("GAU-GAU");  
    }  
}
```

```
public class Gato extends Animal {  
    public void hacerRuido() {  
        System.out.println("MIAUUUU");  
    }  
}
```

```
public class Sirena implements EmiteRuido {  
    public void hacerRuido() {  
        System.out.println("UUUUUUHHHHH");  
    }  
}
```



Interface

```
public class Ciudad {  
    private ArrayList<EmiteRuido> cosas;  
  
    public Ciudad () {  
        cosas = new ArrayList<EmiteRuido>();  
    }  
  
    public void agregar(EmiteRuido e) {  
        cosas.add(e);  
    }  
  
    public void escucharRuidos() {  
        for ( EmiteRuido e : cosas )  
            e.hacerRuido();  
    }  
}
```



Interface

```
public class UsoCiudad {  
  
    public static void main( String [] args ) {  
        Ciudad c = new Ciudad();  
        EmiteRuido r = new Gato(); //Puedo usar como "tipo".  
        c.agregar(r);  
        c.agregar(new Perro());  
        c.agregar(new Humano("Maria"));  
        c.agregar(new Humano("Luis"));  
        c.agregar(new Sirena());  
        c.agregar(new Auto());  
  
        c.escucharRuidos();  
    }  
}
```

Interface -beneficios

- Muy interesante cuando se hace trabajo en equipo.
- Revelar los requerimientos no la implementación. Concepto de encapsulamiento
- La implementación puede cambiar sin afectar a los usuarios
- El que usa la interface no necesita tener la implementación durante el desarrollo.
- Podemos relacionar clases “no conectadas” (como el ejemplo).
- De alguna manera suple la herencia múltiple.

Interface

- Para definir una interface

```
public interface <identificador> {  
    // métodos sin implementación  
}
```

- Una interfaz puede extender otra

```
public interface EmiteRuidoMasFuerte extends EmiteRuido {  
    void hacerRuidoFuerte();  
}
```

- Las interfaces no son parte de la jerarquía de clases

Generics

- Proveen abstracción sobre tipos
 - Son clases, interfaces o métodos que son parametrizados por tipos
- Me permiten lograr implementaciones independientes del tipo de dato “base” a utilizarse
- Muy usado en colecciones de objetos.
- Permite delimitar de manera automática los tipos permitidos en cierta clase, método o interface y facilitar el proceso de chequeo de compilación

Generic - ejemplo

<T> representa un tipo genérico

```
package java.lang
```

```
public interface Comparable<T> {  
    public int compareTo(T o )  
}
```

Generic - Ejemplo

Queremos definir una clase que pueda manejar una lista de elementos y que sean todos del mismo tipo.

```
public class Lista <T> {  
    private T [] lista ; // elementos del tipo T  
    public T get(int i ) // el elemento en la posición i.  
    public void add (T dato ) // agregar dato  
}  
  
Lista<String> ls = new Lista<String>();  
Lista<Empleado> ls = new Lista<Empleado>();
```


Generic

Métodos genéricos

Ejemplo de un método genérico que retorne el nombre de la clase de su parámetro

El parámetro **param** puede ser de cualquier clase denotado por el tipo genérico T.

```
public static <T> String nombreClase ( T param) {  
    return param.getClass().getName();  
}  
  
public static void main(String[] args) {  
    System.out.println(nombreClase("hola"));  
    System.out.println(nombreClase(1));  
    System.out.println(nombreClase(2.0));  
    System.out.println(nombreClase(1.3f));  
    System.out.println(nombreClase('C'));  
    System.out.println(nombreClase(new ArrayList<String>() ));  
    System.out.println(nombreClase(LocalDate.now()));  
}
```

Salida

```
java.lang.String  
java.lang.Integer  
java.lang.Double  
java.lang.Float  
java.lang.Character  
java.util.ArrayList  
java.time.LocalDate
```

Generics

Métodos genéricos

Ejemplo de un método genérico que encuentra el máximo en un arreglo de tipo genérico pero con cierto tipo de restricciones.

wildcard <? super T> que indica que puede ser solo cualquier super clase de T (se restringe).

```
public static <T extends Comparable <? super T>> T findMax( T[]a )
{
    int maxIndex = 0;
    for( int i = 1; i < a.length; i++ )
        if( a[ i ].compareTo( a[ maxIndex ] ) > 0 )
            maxIndex = i;
    return a[ maxIndex ];
}
```

Pruebe en
JSHELL

Generics

```
public static void main( String [ ] args )
{
    String  [ ] sL = { "Joe", "Bob", "Bill", "Zeke" };
    Integer [ ] iL = { 10, 20, 30, 50};
    Double  [ ] fL = {10.20, 10.303, 23.22};

    System.out.println( findMax(sL) );
    System.out.println( findMax(iL) );
    System.out.println( findMax(fL) );
}
```

Generics . Ejemplo de una ED

Lista simplemente enlazada : definimos una lista de tipo genérico

```
public class ListaEnlazada <E> {  
    private class Nodo {  
        E dato;  
        Nodo next;  
        Nodo ( E d ) {  
            this.dato = d;  
        }  
    }  
    private Nodo header = null;  
    public void agregar ( E dato ) {  
        Nodo tmp = header;  
        header = new Nodo (dato);  
        header.next = tmp;  
    }  
  
    public String toString() {  
        String sal = "";  
        Nodo act = header;  
        while (act != null) {  
            sal += act.dato + "\n";  
            act = act.next;  
        }  
        return sal;  
    }  
}
```

Recibe como parámetro un tipo

Dato de tipo genérico

Generics

```
/* Ejemplo de uso de nuestra ListaEnlazada* /  
public static void main ( String [] args) {  
  
    ListaEnlazada<String> LS = new ListaEnlazada<String> ();  
  
    LS.agregar("Hola");  
    LS.agregar("Que tal");  
    LS.agregar("Como estan");  
  
    System.out.print(LS);  
}
```

Ejemplo utilizando Generic

Implementar una clase semejante a la clase del API de java ArrayList, denominada `GenericSimpleArrayList` que soporte cualquier tipo de objeto.

Ejemplo de Generic

```
public class GenericSimpleArrayList<T>
{
    private static final int CAPACIDAD_POR_DEFECTO = 10;

    private int theSize;
    private T [ ] theItems;

    /**
     * Constructor por defecto
     */
    public GenericSimpleArrayList( )
    {
        clear( );
    }
    ...
}
```

Ejemplo de Generic

```
/**
 * Retorna en el número de items en la colección
 */
public int size( )
{
    return theSize;
}

/**
 * Retorna el item en la posición idx
 */
public T get( int idx )
{
    if( idx < 0 || idx >= size( ) )
        throw new ArrayIndexOutOfBoundsException("Idx="+idx+";Size="+size( ) );
    return theItems[ idx ];
}
```


Ejemplo de Generic

```
..  
/**  
 * Cambiar el item de la posición idx. Retorna el valor  
 * anterior  
 */  
public T set( int idx, T newVal )  
{  
    if ( idx < 0 || idx >= size() )  
        throw new ArrayIndexOutOfBoundsException( "Idx="+idx+";size="+size( ) );  
  
    T old = theItems[ idx ];  
    theItems[ idx ] = newVal;  
    return old;  
}  
..
```

Ejemplo de Generic

```
..  
/**  
 * Agregar un item a la colección, al final.  
 */  
public boolean add( T x )  
{  
    if (theItems.length == size() )  
    {  
        T [ ] old = theItems;  
        theItems = (T []) new Object[theItems.length * 2 + 1];  
        for( int i = 0; i < size(); i++ )  
            theItems[i] = old[ i ];  
    }  
  
    theItems[ theSize++ ] = x;  
  
    return true;  
}
```

Ejemplo de Generic

```
..  
/**  
 * Remueve el item de la colección, se desplazan  
 * los componentes.  
 */  
public T remove( int idx )  
{  
    T removedItem = theItems[ idx ];  
  
    for( int i = idx; i < size( ) - 1; i++ )  
        theItems[ i ] = theItems[ i + 1 ];  
  
    theSize--;  
  
    return removedItem;  
}  
..
```

Ejemplo de Generic

```
..  
/**  
 * Inicializa la colección  
 */  
public void clear( )  
{  
    theSize = 0;  
    theItems = (T []) new Object [CAPACIDAD_POR_DEFECTO];  
}  
  
} /* Fin de la clase
```

Ejemplo de Generic

```
public class UsoGenericList {  
  
    public static void main ( String [] args ) {  
        GenericSimpleArrayList<String> sa = new  
            GenericSimpleArrayList<String>();  
        sa.add("Hola");  
        sa.add("Que tal");  
  
        for ( int i = 0 ; i <= sa.size() ; i++)  
            System.out.println(sa.get(i));  
    }  
}
```



Parte IX

Construcciones avanzadas en Java

Expresiones lambda (semejante a Python)

- Funciones que no están asociadas a un determinado nombre y pueden ser argumento de otras funciones.
- En vez de pasar datos ahora podemos pasar también código o cómputo.

Expresiones lambda (ejemplo)

```
public class Calculator {  
    /* Functional interface */  
    interface IntegerMath {  
        int operation(int a, int b);  
    }  
  
    public int operBinary(int a, int b, IntegerMath op) {  
        return op.operation(a, b);  
    }  
  
    public static void main(String... args) {  
        Calculator miCalc = new Calculator();  
  
        IntegerMath suma = (a, b) -> a + b;  
        IntegerMath resta = (a, b) -> a - b;  
  
        System.out.println("40 + 2 = " + miCalc.operBinary(40, 2, suma));  
        System.out.println("20 - 10 = " + miCalc.operBinary(20, 10, resta));  
        System.out.println("20 * 5 = " + miCalc.operBinary(20, 5, (a,b)->a*b ));  
    }  
}
```



```
public class Persona {  
    public enum Sexo {  
        MASC, FEM  
    }  
  
    private String nombre;  
    private LocalDate fecha_nac;  
    private Sexo genero;  
    private String emailAddress;  
  
    public int getEdad() { ... }  
    public Sexo getSexo() { ... }  
    public String getCorreo(){ ... }  
  
    ...  
}
```

```
List<Persona> listaPersona = new List<>();
```

Cree en **Jshell** la clase

Para agregar varias personas:
`listaPersona.add( new Persona(..) )`

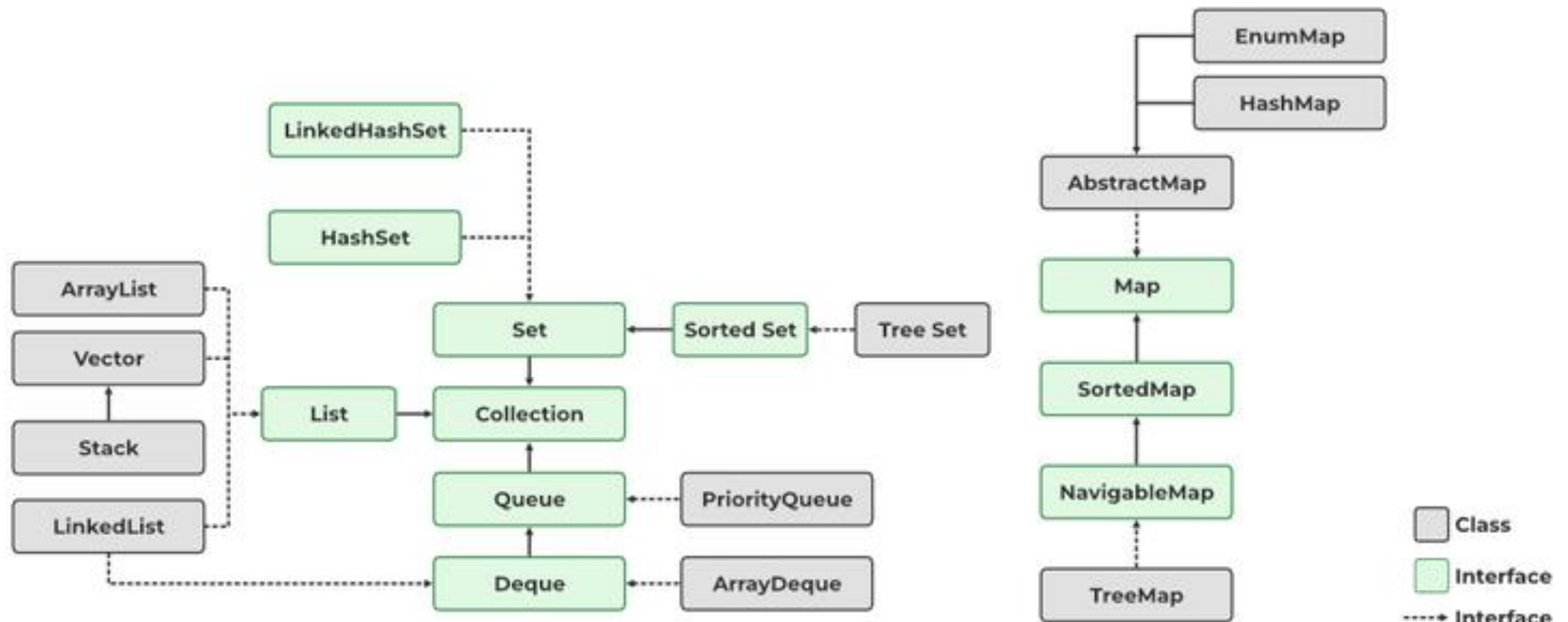
Stream y operaciones agregadas

Tengo una lista de *Persona* en la variable `listaPersona`. *Persona* tiene atributos “Sexo”, “Fec. De Nac”, etc. De esta lista necesito imprimir las direcciones de mail de las personas masculinas entre 18 y 25 años.

```
listaPersona.stream()  
    .filter(  
        p -> p.getSexo() == Persona.Sexo.MASC  
            && p.getEdad() >= 18  
            && p.getEdad() <= 25)  
    .map(p -> p.getCorreo())  
    .forEach(email -> System.out.println(email));
```

Pruebe en **Jshell**.

Colecciones de Java



Subconjunto de colecciones (tomado de <https://www.geeksforgeeks.org/collections-in-java-2/>)

- Conjunto de clases e interfaces que funcionan como una unidad se denomina colección.
- Dos principales Framework de Colecciones existen: ***Collection*** y ***Map*** que son las raíces de clases Java que principalmente nos brinda facilidades de estructura de datos de acuerdo a la necesidad.
- ***Collection***: interface con capacidad de iteración con funcionalidad de agregar datos a la colección.
 - List: colección de objetos ordenados con posibilidad de tener duplicados.
 - Set: colección de objetos sin orden y sin duplicación.
 - Queue: mantiene el orden FIFO.
- ***Map***: interface que soporta clave-valor como datos. No soporta duplicados.

Frameworks de colecciones (versión >=21)

- **Collection Interface:** Collection, Set, Map, Queue, Deque, etc.
- **General-purpose implemations:** HashSet, TreeSet, ArrayList, etc. (nombres de la forma <implementation-style><interface>)
- **Legacy implementations:** Vector, Hashtable
- **Special-purpose implementation**
- **Concurrent implementations**
- **Abstract implementations**
- **Algorithms:** sort, binarySearch, min, max, frequency, disjoint, etc.
- **Infraestructure:** iterator, ordering, runtime exceptions, performance.
- **Arrays utilities:** clase Arrays que tiene varios métodos para manipular arreglos.

<https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/doc-files/coll-reference.html>

General purpose implementations

Interface	Hash Table	Resizable Array	Balanced Tree	Linked List	Hash Table + Linked List
Set	HashSet		TreeSet		LinkedHashSet
List		ArrayList		LinkedList	
Queue, Deque		ArrayDeque		LinkedList	
Map	HashMap		TreeMap		LinkedHashMap

<https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/doc-files/coll-overview.html>

Bibliografía

- Existen muchos libros de Java, algunos en español:
 - Flanagan, David. Java in a NutShell, 5th edition. O'Reilly. 2005. (En biblioteca edición antigua en español).
 - Deitel H. y Deitel P. Como programar en Java. Pearson Education. 2008 (en Biblioteca).
 - Dean J y Dean R. Introducción a la programación con JAVA. 2009 (en Biblioteca)
 - Introduction to Programming Using Java. David J. Eck. 2022.
<https://math.hws.edu/javanotes/>
 - Introduction to Programming in Java. An interdisciplinary Approach. R. Sedgewick & K. Wayne. 2015. <https://introcs.cs.princeton.edu/java/home/>

Algunas fuentes de inicio de uso del lenguaje

<https://docs.oracle.com/javase/tutorial/>

<https://www.w3schools.com/java/>

<https://www.tutorialspoint.com/java/index.htm>

<https://docs.oracle.com/en/java/javase/25/books.html>

<https://dev.java/learn/>

Especificación del lenguaje (versión 25):

<http://docs.oracle.com/javase/specs/jls/se25/html/index.html>